COMSATS University Islamabad, Pakistan

PneumoAI

A project presented to

COMSATS Institute of Information Technology, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Fatima Sardar **CIIT/FA22-BCS-282/SWL**

Ujala Shoukat **CIIT/FA22-BCS-160/SWL**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Fatima Sardar **Ujala Shoukat**

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) **PneumoAI** was Developed by **Fatima Sardar CIIT/FA22-BCS-282/SWL** and **Ujala Shoukat CIIT/FA22-BCS-160/SWL** under the supervision of **Dr. Yawar Abbas Abid** and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science

---

Supervisor
Dr Yawar Abbas Abid
Professor
Department of Computer Science

External Examiner
Dr Muhammad Sulman Bashir
Assistant Professor
Department of CS & IT
Virtual University of Pakistan

Dr Zafar Iqbal Roy
Assistant Professor,
Head of Department
Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Pneumonia remains one of the leading causes of mortality worldwide, particularly in regions where access to timely and accurate medical diagnosis is limited. Traditional diagnosis relies heavily on radiologists to interpret chest X-ray images, a process that is time-consuming, resource-intensive, and prone to human error. Furthermore, patients often lack reliable and accessible information regarding disease management, recovery, and preventive care.

To address these challenges, this project presents **PneumoAI**, an intelligent web-based system designed to assist in the early detection of pneumonia using Artificial Intelligence. The system integrates a Convolutional Neural Network (CNN) model trained on chest X-ray datasets to classify images as either *Normal* or *Pneumonia* with an associated confidence score. In addition to automated diagnosis, the system incorporates a Large Language Model (LLM)-based AI health assistant that provides context-aware, pneumonia-specific guidance to users.

The proposed system is implemented using a modern full-stack architecture. The frontend is developed using Next.js and Tailwind CSS to ensure a responsive and user-friendly interface. The backend is built using Node.js and Express.js, which manages authentication, API communication, and business logic. A Flask-based microservice hosts the CNN model for image analysis, while PostgreSQL is used for structured data storage and Cloudinary for secure image storage. The AI assistant is integrated through an external LLM API, ensuring controlled and relevant responses.

PneumoAI offers several key features, including secure user authentication, X-ray image upload and analysis, result visualization with confidence scores, historical record management, PDF report generation, and an interactive chatbot for health-related queries. The system follows an Agile development methodology, enabling iterative improvements, continuous testing, and efficient integration of AI components.

This project demonstrates how the integration of deep learning and web technologies can enhance healthcare accessibility and efficiency. While PneumoAI is not intended to replace professional medical diagnosis, it serves as a supportive tool for early detection and patient education. The system contributes toward reducing diagnostic delays and improving awareness, particularly in underserved regions, thereby showcasing the potential of AI-driven healthcare solutions.

In conclusion, PneumoAI highlights the potential of AI-driven solutions in transforming healthcare systems, particularly in underserved regions. By reducing diagnostic delays, improving accessibility, and providing meaningful user guidance, the system contributes toward more efficient and informed healthcare practices. The project reflects a successful application of interdisciplinary knowledge and sets a foundation for future advancements in intelligent healthcare systems.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor **Dr. Yawar Abbas Abid**. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Fatima Sardar **Ujala Shoukat**

---------- ----------

Abbreviations

CNN	Convolutional Neural Networks
LLM	Large Language Model
HCI	Human-Computer Interaction
NPL	Natural Language Processing
ML	Machine Learning

Table of Contents

List of Figures

List of Tables

CHAPTER # 1

1. Introduction

This chapter presents a comprehensive overview of the PneumoAI project, outlining its purpose, background, and significance within the rapidly evolving intersection of healthcare and technology. In recent years, the integration of intelligent systems into medical practice has transformed how diseases are diagnosed, monitored, and managed. Among these advancements, the application of Artificial Intelligence particularly in medical imaging has shown promising potential in improving diagnostic accuracy, reducing human error, and enhancing clinical efficiency. The PneumoAI project is developed within this context, aiming to leverage advanced computational techniques to assist in the early detection of pneumonia, a potentially life-threatening respiratory condition.

Pneumonia remains a major global health concern, especially in developing regions where access to experienced radiologists and timely diagnosis may be limited. Traditional diagnostic methods, such as chest X-rays interpreted by medical professionals, are effective but can be time-consuming and subject to variability in interpretation. This creates a critical need for automated and reliable systems that can assist healthcare providers in making faster and more accurate decisions. The PneumoAI project addresses this challenge by introducing a web-based platform that integrates deep learning models for image classification with conversational AI capabilities for user interaction and support.

At its core, the project utilizes techniques from Deep Learning, a specialized branch of machine learning that focuses on neural networks with multiple layers capable of learning complex patterns from data. By training these models on medical imaging datasets, the system is designed to identify patterns associated with pneumonia in chest X-ray images. This not only aids in detection but also reduces the burden on healthcare professionals by providing preliminary diagnostic insights. In addition, the integration of a conversational interface enhances user experience by enabling intuitive communication, allowing users to interact with the system, ask questions, and receive guidance in a user-friendly manner.

Furthermore, this chapter highlights the academic relevance of the PneumoAI project, particularly in relation to coursework in computer science, data science, and healthcare informatics. It demonstrates the practical application of theoretical concepts such as supervised learning, model evaluation, and system integration within a real-world problem domain. By combining technical implementation with healthcare objectives, the project serves as a multidisciplinary solution that aligns with current industry trends and academic expectations.

In addition to introducing the system, this chapter provides an overview of existing research and similar systems developed in the field of AI-based medical diagnosis. Various studies have explored the use of deep learning models for detecting pneumonia and other lung diseases; however, many of these systems face limitations such as lack of generalizability, insufficient dataset diversity, or absence of user-friendly interfaces. A critical analysis of these existing solutions is presented to identify gaps and justify the need for the proposed system. This analysis forms the basis for defining the unique contributions of PneumoAI.

Moreover, the chapter outlines the software development methodology adopted for this project, emphasizing a structured approach to system design, implementation, and evaluation. The chosen development lifecycle ensures that the project is built systematically, with careful consideration given to

requirements analysis, model development, testing, and deployment. The rationale behind selecting this methodology is also discussed, highlighting its suitability for managing both the technical and research-oriented aspects of the project.

Overall, this introduction establishes a strong foundation for understanding the problem domain, the technological approach, and the significance of the proposed solution. It sets the stage for subsequent chapters by clearly defining the objectives, scope, and direction of the PneumoAI project, while also emphasizing its contribution to both academic research and practical healthcare applications.

1.1. Brief

PneumoAI is an intelligent, web-based healthcare application designed to assist in the early detection and preliminary assessment of pneumonia through the analysis of chest X-ray images. The primary objective of the system is to support medical professionals and users by providing a fast, accessible, and reliable diagnostic aid that leverages advancements in Artificial Intelligence. By automating the interpretation of medical images, the system aims to reduce diagnostic delays and improve decision-making efficiency, particularly in environments where expert radiological support may be limited.

At the core of PneumoAI lies a deep learning-based image classification model built using Convolutional Neural Networks (CNNs). These networks are specifically designed to process visual data and extract hierarchical features from images, making them highly effective for medical imaging tasks. The model is trained on a dataset of labeled chest X-ray images and is capable of classifying inputs into two primary categories: *Normal* and *Pneumonia*. In addition to categorical predictions, the system provides a confidence score that quantifies the certainty of the model's output, thereby enhancing transparency and allowing users to better interpret the results. This probabilistic output is particularly important in healthcare contexts, where uncertainty must be clearly communicated rather than hidden.

Beyond automated image classification, PneumoAI incorporates an intelligent conversational interface powered by a Large Language Model (LLM). This AI assistant extends the functionality of the system by offering contextual information related to pneumonia, including its common symptoms, possible causes, preventive measures, and recommended recovery practices. The integration of such a conversational component transforms the system from a purely diagnostic tool into an interactive support platform, enabling users to engage with the system in a more intuitive and informative manner. This is particularly beneficial for non-technical users who may require additional guidance alongside diagnostic outputs.

From a system design perspective, PneumoAI is developed using a modern full-stack architecture that emphasizes scalability, modularity, and maintainability. The frontend of the application is built using Next.js, a powerful React-based framework that supports server-side rendering and optimized performance. This is complemented by Tailwind CSS, which enables efficient and responsive user interface design through a utility-first styling approach. Together, these technologies ensure a seamless and visually appealing user experience across different devices and screen sizes.

On the backend, the system is implemented using Node.js and Express.js, which facilitate the development

of a robust and scalable server-side application. These technologies handle API requests, user interactions, and communication between different system components. The deep learning model is deployed separately as a microservice using Flask, allowing for efficient handling of model inference requests and enabling independent scaling of the AI component. This microservices-based approach enhances system flexibility and simplifies future upgrades or modifications to the model.

For data management, PneumoAI utilizes PostgreSQL, a reliable and feature-rich relational database system that supports structured storage of user data, system logs, and prediction records. The choice of PostgreSQL ensures data integrity, consistency, and efficient query performance, which are critical for maintaining a dependable healthcare application.

The development of PneumoAI follows the principles of Agile methodology, which promotes iterative progress, continuous feedback, and adaptive planning. This approach allows the development team to refine system features incrementally, conduct regular testing, and integrate improvements based on evaluation results. Agile practices are particularly suitable for projects involving AI components, as model performance and system requirements often evolve over time.

This report provides a detailed account of the entire system development process, including an exploration of the problem domain, identification of system requirements, architectural design decisions, implementation strategies, and evaluation methodologies. It also highlights the challenges encountered during development and the solutions adopted to address them. Through this comprehensive documentation, the report demonstrates how PneumoAI contributes to the growing field of AI-driven healthcare solutions by combining technical innovation with practical usability.

1.2. Relevance to Course Modules

The development of PneumoAI demonstrates a strong alignment with several core modules studied during the Bachelor of Computer Science program. The project serves as a practical implementation of theoretical knowledge, illustrating how concepts from different domains can be integrated to solve a real-world healthcare problem. Each module contributes distinct principles and techniques that collectively support the design, development, and deployment of the system.

Firstly, the module of Artificial Intelligence and Machine Learning plays a central role in the project. Concepts such as supervised learning, feature extraction, model training, and evaluation are directly applied in building the pneumonia detection system. In particular, the use of Convolutional Neural Networks (CNNs) demonstrates the application of deep learning techniques for image classification tasks. The model is trained on labeled chest X-ray datasets to learn patterns associated with pneumonia, enabling it to make predictions on unseen data. Additionally, performance evaluation metrics such as accuracy, precision, and loss functions are utilized to assess and improve model effectiveness, reflecting key learning outcomes from this module.

Secondly, the Database Systems module is applied through the integration of a relational database management system. PneumoAI uses PostgreSQL to store and manage structured data, including user

profiles, prediction history, and interaction logs generated by the AI assistant. Concepts such as database design, normalization, schema creation, and query optimization are implemented to ensure data consistency, integrity, and efficient retrieval. This demonstrates how theoretical database principles can be translated into a reliable backend data management solution.

The project also heavily incorporates knowledge from the Web Development module. The frontend of the system is developed using Next.js, which enables server-side rendering and improved performance, while Tailwind CSS is used to design a responsive and visually consistent user interface. On the backend, Node.js and Express.js are utilized to build RESTful APIs that handle communication between the client interface and server-side components. This reflects a full-stack development approach, where both client-side and server-side technologies are integrated to create a cohesive web application.

In addition, the principles of Software Engineering are applied throughout the project lifecycle. The development process includes requirement gathering, system analysis, architectural design, implementation, testing, and deployment. Tools such as UML diagrams are used to model system behavior and structure, providing a clear blueprint for development. Furthermore, the adoption of an Agile methodology enables iterative progress, continuous feedback, and flexibility in handling changes. This ensures that the system evolves systematically while maintaining quality and reliability.

Another important aspect of the project is its alignment with the Human-Computer Interaction (HCI) module. The design of PneumoAI emphasizes usability, accessibility, and user experience. The interface is structured to be intuitive and easy to navigate, ensuring that users regardless of their technical background can interact with the system effectively. Considerations such as layout design, responsiveness, clarity of information, and user feedback mechanisms are incorporated to enhance overall usability. The inclusion of a conversational AI assistant further improves interaction by allowing users to communicate naturally with the system.

Overall, PneumoAI exemplifies the practical integration of multiple academic disciplines into a unified system. It highlights how theoretical concepts learned across different modules can be combined to address a complex, real-world problem in healthcare. This interdisciplinary approach not only strengthens technical competence but also demonstrates the ability to design and implement innovative solutions that have meaningful societal impact.

1.3. Project Background

Pneumonia is a serious respiratory infection that primarily affects the air sacs in the lungs, causing them to fill with fluid or pus, which leads to symptoms such as cough, fever, difficulty breathing, and chest pain. If not diagnosed and treated promptly, pneumonia can result in severe complications, including respiratory failure and even death. According to global health statistics, pneumonia continues to be one of the leading causes of mortality worldwide, particularly among vulnerable populations such as children under five years of age and elderly individuals. This highlights the urgent need for timely and accurate diagnostic solutions that can be deployed at scale.

Traditionally, the diagnosis of pneumonia relies heavily on clinical examination and the interpretation of chest X-ray images by trained radiologists. While this method is widely accepted and effective, it presents several practical challenges that limit its efficiency and accessibility. One of the major issues is the limited availability of skilled radiologists, especially in rural and under-resourced regions where healthcare infrastructure is often inadequate. This shortage creates delays in diagnosis and treatment, potentially worsening patient outcomes.

In addition to accessibility concerns, the process of manual image interpretation is inherently time-consuming. Radiologists are required to carefully analyze each image, identify abnormal patterns, and make diagnostic decisions based on their expertise. In high-demand healthcare environments, such as busy hospitals, this workload can become overwhelming, increasing the likelihood of delays. Furthermore, despite their expertise, human interpretation is not entirely free from error. Factors such as fatigue, subjective judgment, and variability in experience levels can lead to inconsistencies in diagnosis. These limitations emphasize the need for automated systems that can assist in improving both the speed and accuracy of medical image analysis.

In recent years, advancements in Artificial Intelligence have opened new possibilities in the field of healthcare, particularly in medical imaging. One of the most significant developments is the application of Deep Learning techniques, which enable computers to learn complex patterns directly from large datasets. Among these techniques, Convolutional Neural Networks (CNNs) have demonstrated exceptional performance in image classification tasks. CNNs are capable of automatically extracting relevant features from images, such as textures, shapes, and edges, which are crucial for identifying abnormalities in medical scans. Numerous studies have shown that CNN-based models can achieve high accuracy in detecting pneumonia from chest X-ray images, making them a suitable choice for developing automated diagnostic tools.

Parallel to these advancements in image analysis, the emergence of Large Language Models (LLMs) has transformed the way users interact with intelligent systems. LLMs are capable of understanding natural language queries and generating contextually relevant responses, enabling the development of conversational AI systems. In the context of healthcare, such systems can provide users with valuable information regarding symptoms, preventive measures, treatment options, and general health guidance. This is particularly beneficial for individuals who may not have immediate access to medical professionals but require reliable information.

The convergence of deep learning for image analysis and LLMs for conversational interaction presents a unique opportunity to develop integrated healthcare solutions. Rather than functioning as isolated tools, these technologies can be combined to create systems that not only perform automated diagnosis but also enhance user understanding and engagement. This dual capability is essential in modern healthcare applications, where both accuracy and accessibility play critical roles.

PneumoAI is developed within this context as an innovative solution aimed at addressing the existing gaps in pneumonia diagnosis and patient support. By integrating AI-based image classification with an interactive conversational assistant, the system provides both diagnostic insights and educational guidance. This approach not only improves the efficiency of disease detection but also empowers users with

knowledge, enabling them to make informed decisions regarding their health.

In summary, the background of this project is rooted in the need to overcome the limitations of traditional diagnostic methods through the application of advanced AI technologies. PneumoAI represents a step toward more accessible, efficient, and user-centered healthcare systems, aligning with the broader trend of digital transformation in the medical field.

1.4. Literature Review

The application of Artificial Intelligence in healthcare, particularly in medical imaging, has gained significant attention in recent years. Researchers and organizations have explored various approaches to automate disease detection, improve diagnostic accuracy, and enhance healthcare accessibility. This section reviews key studies and existing systems related to pneumonia detection and AI-driven healthcare solutions, while also identifying their limitations.

One of the most influential contributions in this domain is the work by Pranav Rajpurkar et al. (2017), who introduced **CheXNet**, a deep learning model designed to detect pneumonia from chest X-ray images. The model is based on a 121-layer convolutional neural network and was trained on the ChestX-ray14 dataset. Their study demonstrated that the model achieved performance comparable to practicing radiologists in detecting pneumonia. This work was significant because it validated the potential of Deep Learning techniques in medical diagnosis and highlighted how automated systems could assist healthcare professionals. However, despite its high accuracy, CheXNet was primarily developed as a research prototype and did not focus on real-world deployment challenges such as user interaction, scalability, or integration with clinical workflows [1].

Another important study was conducted by Daniel S. Kermany et al. (2018), who applied deep learning methods to classify chest X-ray images for pneumonia detection. Their work further confirmed the effectiveness of Convolutional Neural Networks (CNNs) in identifying disease patterns in medical images. The model achieved high accuracy and demonstrated the feasibility of using AI systems as diagnostic tools. Nevertheless, similar to earlier research, the study primarily focused on model performance and lacked emphasis on practical implementation aspects such as system usability, real-time processing, and integration into healthcare systems [2].

In addition to academic research, several commercial solutions have emerged in this field. One notable example is Qure.ai, which offers AI-powered radiology tools for automated diagnosis of chest diseases, including pneumonia. These systems are designed for deployment in hospitals and diagnostic centers, providing fast and reliable image analysis. While such platforms are highly effective and clinically validated, they are often expensive and require specialized infrastructure, making them less accessible for educational purposes, small healthcare facilities, or independent developers. This highlights a key limitation in terms of accessibility and scalability across different contexts [3].

Furthermore, platforms such as Kaggle provide datasets and pretrained models for pneumonia detection, enabling researchers and students to experiment with deep learning techniques. Many implementations

available on Kaggle demonstrate strong model performance and serve as valuable learning resources. However, these solutions are typically confined to experimental or research environments. They often lack essential features required for real-world applications, such as user interfaces, deployment pipelines, system integration, and user interaction capabilities. As a result, there exists a gap between research prototypes and fully functional applications [4].

In parallel with advancements in image-based diagnosis, recent studies have explored the use of AI-driven conversational systems in healthcare. These systems utilize techniques from Natural Language Processing (NLP) to enable interaction between users and machines through natural language. Healthcare chatbots have been developed to provide information about symptoms, suggest preventive measures, and guide users toward appropriate medical actions. While these systems enhance accessibility to healthcare information, they are often developed independently of diagnostic tools, limiting their ability to provide integrated support.

A critical analysis of the existing literature reveals that, although significant progress has been made in applying AI to pneumonia detection, several limitations persist. Many research studies prioritize model accuracy while overlooking factors such as usability, accessibility, and system integration. Commercial solutions, while effective, are not always affordable or widely accessible. Similarly, research-based implementations often lack deployment and interaction capabilities, reducing their practical applicability.

These gaps highlight the need for a comprehensive system that not only performs accurate disease detection but also integrates user-friendly interfaces and interactive support mechanisms. PneumoAI addresses these limitations by combining deep learning-based image classification with a conversational AI assistant within a single web-based platform. This integrated approach aims to bridge the gap between research and real-world application, providing both diagnostic functionality and user-centered interaction in an accessible and scalable manner.

1.5. Analysis from Literature Review

The review of existing studies and systems clearly indicates that, although significant advancements have been made in applying Artificial Intelligence to healthcare, most solutions remain limited in scope and practical usability. A critical analysis of the literature reveals that current approaches tend to focus on isolated functionalities rather than delivering a comprehensive, integrated system. Specifically, many solutions either emphasize accurate disease detection or provide informational support, but rarely combine both capabilities within a unified and user-friendly platform.

For instance, research-oriented models such as CheXNet demonstrate the effectiveness of Deep Learning techniques in medical image classification. These models, often built using Convolutional Neural Networks (CNNs), achieve high accuracy in detecting pneumonia from chest X-ray images. While this represents a major technological achievement, such models are typically developed in controlled research environments and lack essential features required for real-world application. They do not include user interfaces, system integration, or deployment mechanisms, making them inaccessible to non-technical users and limiting their practical impact in healthcare settings.

On the other hand, commercial platforms such as Qure.ai provide more comprehensive solutions by integrating AI-based diagnostic tools into clinical workflows. These systems are designed for real-world use and offer features such as automated radiology reporting and decision support. However, their adoption is often constrained by high costs, infrastructure requirements, and limited accessibility. As a result, such solutions are not easily available to students, researchers, or small healthcare providers, particularly in developing regions where cost and resource limitations are significant barriers.

Similarly, implementations available on platforms like Kaggle provide valuable resources for experimentation and learning. These implementations often include pretrained models and datasets that allow users to explore pneumonia detection using deep learning. Despite their educational value, these solutions are generally confined to experimental environments. They lack critical components such as user authentication, data management systems, reporting features, and interactive interfaces. Consequently, they do not meet the requirements of a fully functional application intended for end users.

Another important observation from the literature is the growing use of conversational AI systems in healthcare. These systems, often powered by Natural Language Processing, are capable of providing users with information about symptoms, preventive measures, and general health advice. While they enhance accessibility to healthcare knowledge, they are usually developed as standalone tools and are not integrated with diagnostic systems. This separation limits their effectiveness, as users must rely on multiple platforms to obtain both diagnostic insights and informational support.

The analysis of these existing solutions highlights several key gaps:

- **Lack of integration:** Most systems do not combine diagnostic capabilities with interactive user support.
- **Limited accessibility:** Commercial solutions are often expensive and not widely available.
- **Absence of user-centric design:** Research models lack interfaces that cater to general users.
- **Restricted functionality:** Experimental implementations do not include essential features such as data management, reporting, and system interaction.

PneumoAI is specifically designed to address these limitations by adopting a more holistic and user-centered approach. The system integrates a CNN-based pneumonia detection model into a complete web application, enabling users to upload chest X-ray images and receive diagnostic predictions in real time. In addition to this, the inclusion of an AI-powered conversational assistant enhances the system's functionality by providing educational support related to pneumonia, including symptoms, prevention strategies, and recovery guidance.

Furthermore, PneumoAI incorporates essential application-level features such as user authentication, prediction history tracking, and report generation. These features ensure that the system is not only technically capable but also practically usable in real-world scenarios. The emphasis on accessibility and ease of use allows the system to cater to a broader audience, including students, researchers, and individuals without specialized medical or technical knowledge.

Another key strength of the proposed system lies in its integration of multiple technologies within a single

platform. By combining deep learning-based image analysis with conversational AI and modern web development frameworks, PneumoAI delivers a unified solution that bridges the gap between research prototypes and real-world applications. This integration enhances both functionality and user experience, making the system more effective and impactful.

In conclusion, the analysis of the literature demonstrates that while existing solutions have made valuable contributions to the field of AI-driven healthcare, they often fall short in terms of integration, accessibility, and usability. PneumoAI addresses these shortcomings by providing a comprehensive, user-centric, and accessible platform that combines automated disease detection with interactive user support. This positions the proposed system as a more practical and holistic solution compared to existing approaches.

1.6. Methodology and Software Lifecycle for This Project

The development of PneumoAI follows the principles of the Agile methodology, a widely adopted software development approach that emphasizes flexibility, collaboration, and iterative progress. Unlike traditional linear models such as the Waterfall model, Agile allows continuous refinement of system requirements and supports incremental delivery of functional components. This makes it particularly suitable for projects like PneumoAI, where both system requirements and AI model performance may evolve during development.

In this project, the Software Development Lifecycle (SDLC) is structured into multiple iterations, commonly referred to as sprints. Each sprint focuses on the development and refinement of a specific module or feature of the system. This modular approach ensures that the system is built progressively, allowing individual components to be tested, evaluated, and improved before moving to the next stage. The major modules identified for development include user authentication, image upload and processing, CNN model integration, AI chatbot integration, and user interface enhancements.

The first module, user authentication, is designed to ensure secure access to the system. It involves implementing features such as user registration, login, and session management. This module is critical for maintaining data privacy and enabling personalized features such as prediction history tracking. The second module focuses on image upload and processing, where users can submit chest X-ray images for analysis. This stage involves validating input data, preprocessing images, and preparing them for model inference.

A core component of the system is the integration of the deep learning model based on Convolutional Neural Networks (CNNs). This module handles the interaction between the web application and the AI model, ensuring that images are processed efficiently and predictions are returned in real time. The deployment of the model as a separate microservice enhances scalability and allows independent updates to the AI component without affecting the overall system.

Another significant module is the integration of the conversational AI assistant powered by a Large Language Model (LLM). This component enables users to interact with the system through natural language, providing information about pneumonia, its symptoms, preventive measures, and recovery

guidelines. The inclusion of this module enhances user engagement and transforms the system into a more comprehensive healthcare support platform.

In addition to functional modules, continuous improvements are made to the user interface and overall user experience. This includes refining layout design, ensuring responsiveness across devices, and improving accessibility. These enhancements are guided by principles from Human-Computer Interaction, ensuring that the system remains intuitive and easy to use for a wide range of users.

Each sprint in the Agile lifecycle follows a structured process consisting of four key phases: requirement analysis, design, implementation, and testing. During the requirement analysis phase, specific objectives for the sprint are identified based on system needs and feedback from previous iterations. This is followed by the design phase, where system architecture, workflows, and data structures are planned. The implementation phase involves the actual development of features, including coding and integration of components. Finally, the testing phase ensures that the developed features function correctly, meet quality standards, and are free from critical errors.

One of the key advantages of adopting Agile in this project is the ability to incorporate continuous feedback. Regular evaluation of each sprint allows issues to be identified and addressed early in the development process, reducing the risk of major errors in later stages. Additionally, this iterative approach supports gradual enhancement of system features, enabling the development team to refine both the AI model and the application interface over time.

Furthermore, Agile promotes adaptability, which is particularly important in AI-based projects where model performance and system requirements may change based on experimental results. By allowing flexibility in planning and execution, the methodology ensures that the final system is both technically robust and aligned with user needs.

In conclusion, the use of the Agile Software Development Lifecycle provides a structured yet flexible framework for the development of PneumoAI. It supports incremental development, continuous testing, and iterative improvement, ensuring that the system evolves effectively throughout the project lifecycle. This approach not only enhances the quality and reliability of the final product but also ensures that it meets both technical and user-centric requirements.

1.7. Rationale behind Selected Methodology

The selection of an appropriate software development methodology is a critical factor in determining the success of any project, particularly one that involves multiple components and evolving requirements. For the development of PneumoAI, the Agile methodology was chosen due to its flexibility, adaptability, and strong alignment with the dynamic nature of AI-based systems. Unlike traditional methodologies such as the Waterfall model, which follow a rigid sequential structure, Agile supports iterative development and continuous improvement, making it more suitable for complex and exploratory projects.

One of the primary reasons for selecting Agile is its inherent flexibility. AI systems, especially those based on Deep Learning, require continuous experimentation, tuning, and optimization. The performance of

models such as Convolutional Neural Networks (CNNs) depends on various factors, including dataset quality, hyperparameter configuration, and training techniques. As a result, it is often necessary to refine the model multiple times throughout the development process. Agile facilitates this by allowing iterative updates and adjustments without requiring a complete redesign of the system.

Another key advantage of Agile is its support for incremental and modular development. PneumoAI is composed of several independent yet interconnected components, including the frontend interface, backend services, machine learning model, and conversational AI module. Developing these components simultaneously in a structured manner can be challenging under traditional methodologies. Agile addresses this by dividing the project into manageable sprints, each focusing on a specific module or feature. This modular approach enables efficient development, easier debugging, and seamless integration of different system components.

Continuous testing is another significant factor that influenced the choice of methodology. In AI-driven applications, it is not sufficient to test only the software functionality; the performance and reliability of the AI model must also be evaluated regularly. Agile promotes frequent testing at every stage of development, ensuring that both system components and prediction outputs meet the required standards. This helps in identifying errors early, improving system stability, and maintaining the overall quality of the application.

Adaptability is also a crucial consideration, particularly for a project that may evolve over time. During the development of PneumoAI, additional features such as PDF report generation, enhanced chatbot capabilities, and improved user interface elements may be introduced based on feedback and evaluation. Agile allows such changes to be incorporated in later iterations without disrupting the existing system structure. This ensures that the project remains responsive to new requirements and technological advancements.

Furthermore, Agile contributes to effective risk management. By prioritizing the development of core functionalities such as the pneumonia detection model and prediction system in the early stages, the project minimizes the risk of failure. Early implementation and testing of critical components provide valuable insights into potential challenges, allowing corrective actions to be taken promptly. This proactive approach reduces uncertainties and increases the likelihood of successful project completion.

In addition to these technical advantages, Agile encourages continuous feedback and collaboration, which are essential for refining both system functionality and user experience. Regular evaluation of each sprint ensures that the project remains aligned with its objectives and that improvements can be made based on observed outcomes.

In conclusion, the Agile methodology is well-suited for the development of PneumoAI due to its flexibility, support for incremental development, emphasis on continuous testing, adaptability to changing requirements, and ability to reduce project risks. Its iterative nature aligns effectively with the needs of an AI-driven, multi-component system, ensuring that the final product is both robust and responsive to user needs.

CHAPTER # 2

2. Problem Definition

This chapter provides a comprehensive examination of the problem addressed by the PneumoAI system, focusing on the limitations of traditional diagnostic approaches and the growing need for intelligent, technology-driven solutions in healthcare. The early and accurate detection of Pneumonia remains a significant challenge, particularly in environments where medical expertise and resources are limited. Despite advancements in healthcare infrastructure, many diagnostic processes still rely heavily on manual interpretation and human expertise, which can introduce inefficiencies and inconsistencies.

In conventional clinical settings, pneumonia diagnosis is typically performed through the analysis of chest X-ray images by trained radiologists. While this method is considered reliable, it is not without its limitations. The increasing number of patients, combined with a shortage of qualified radiologists in certain regions, creates a bottleneck in the diagnostic process. This issue is especially prominent in rural or underdeveloped areas, where access to specialized healthcare professionals is often restricted. As a result, delays in diagnosis can occur, leading to delayed treatment and increased risk of complications.

Moreover, the manual interpretation of medical images is inherently subjective and may vary depending on the experience and workload of the radiologist. Factors such as fatigue, time pressure, and varying levels of expertise can contribute to diagnostic errors or inconsistencies. These challenges highlight the need for systems that can support healthcare professionals by providing consistent, fast, and reliable diagnostic assistance.

With the rapid advancement of Artificial Intelligence, new opportunities have emerged to address these challenges. AI-driven systems, particularly those based on Deep Learning, have demonstrated strong capabilities in analyzing medical images and identifying patterns that may not be easily detectable by the human eye. These technologies have the potential to significantly improve diagnostic accuracy while reducing the time required for analysis.

However, despite the availability of advanced AI models, many existing solutions are either limited to research environments or designed for large-scale commercial use, making them inaccessible to a wider audience. There is a noticeable gap between highly accurate AI models and user-friendly applications that can be easily adopted in practical scenarios. Additionally, most systems focus solely on diagnosis and do not provide users with supporting information or guidance, which is essential for improving understanding and decision-making.

The PneumoAI system is developed to address these challenges by providing an integrated, intelligent solution for pneumonia detection. It combines automated image analysis with an interactive user interface and a conversational AI assistant, enabling both diagnosis and user education within a single platform. This approach not only improves accessibility but also enhances user engagement and understanding.

This chapter further outlines the expected deliverables of the project, including a fully functional web-based application, an integrated AI model for image classification, and a conversational assistant for user interaction. It also defines the system requirements, including both functional and non-functional aspects, to ensure that the proposed solution meets user needs and performance standards.

In addition, a comparison with existing systems is presented to justify the necessity of the proposed approach. By identifying the limitations of current solutions and demonstrating how PneumoAI addresses these gaps, this chapter establishes a clear rationale for the development of the system.

Overall, this chapter lays the foundation for understanding the core problem, its real-world implications, and the need for an innovative, AI-driven solution. It provides a structured framework for analyzing system requirements and guiding the design and implementation of PneumoAI in subsequent sections.

2.1. Problem Statement

Pneumonia is a serious and potentially life-threatening respiratory condition that requires timely and accurate diagnosis to prevent severe complications and reduce mortality rates. Early detection plays a crucial role in effective treatment; however, the current diagnostic process presents several limitations that hinder efficiency, accessibility, and reliability. These challenges are particularly evident in healthcare systems that rely heavily on manual interpretation and limited medical resources.

Traditionally, pneumonia diagnosis involves the analysis of chest X-ray images by trained radiologists. While this method is clinically effective, it is not always efficient or universally accessible. One of the primary issues is the shortage of skilled radiologists, especially in rural and underdeveloped regions. This lack of expertise leads to delays in diagnosis, which can significantly impact patient outcomes. In time-sensitive conditions such as pneumonia, even minor delays can result in disease progression and increased risk of complications.

In addition to accessibility challenges, the manual interpretation of X-ray images is inherently time-consuming. Radiologists must carefully examine each image, identify abnormalities, and make diagnostic decisions based on their experience and judgment. In high-demand environments, such as busy hospitals, this workload can become overwhelming. As a result, the quality and consistency of diagnosis may be affected. Human error, influenced by factors such as fatigue, workload pressure, and subjective interpretation, can lead to misdiagnosis or missed diagnoses. These issues highlight the need for systems that can assist in improving both speed and accuracy.

Another critical limitation of existing diagnostic systems is their lack of accessibility for general users. Most available solutions are designed for clinical use and require specialized knowledge to operate. This restricts their usability to healthcare professionals, leaving patients with limited access to diagnostic tools and information. In many cases, individuals must rely entirely on healthcare facilities, which may not always be readily available.

Furthermore, there is a significant gap in providing reliable and user-friendly educational support to patients. After receiving a diagnosis, patients often seek information about symptoms, treatment options, preventive measures, and recovery practices. However, the information available through traditional sources or online platforms is often fragmented, inconsistent, or difficult to understand. Existing systems typically focus either on disease detection or on providing general health information, but rarely integrate both functionalities into a single, cohesive platform.

With advancements in Artificial Intelligence, particularly in Deep Learning, there is an opportunity to address these challenges through automation and intelligent system design. AI-based models, such as Convolutional Neural Networks (CNNs), have demonstrated strong capabilities in analyzing medical images and detecting diseases with high accuracy. However, many existing implementations remain confined to research environments or lack integration with user-friendly applications.

Therefore, there is a clear need for an intelligent, accessible, and integrated system that not only performs accurate disease detection but also supports users with meaningful and understandable information. The proposed solution should address both technical and user-centric challenges by combining diagnostic capabilities with interactive support.

Specifically, the system should be capable of:

- Automatically detecting pneumonia from chest X-ray images with high accuracy and reliability
- Providing users with clear and interpretable results, including confidence levels to indicate prediction certainty
- Offering interactive, context-aware health guidance through an AI-powered assistant
- Ensuring accessibility and ease of use for both technical and non-technical users

The PneumoAI system is proposed as a solution to these challenges. It integrates deep learning-based image analysis with a conversational AI assistant within a web-based platform. By combining automated diagnosis with user-friendly interaction and educational support, the system aims to enhance both the efficiency and accessibility of pneumonia detection. This integrated approach addresses the limitations of existing systems and provides a more comprehensive solution for modern healthcare needs.

2.2. Deliverables and Development Requirements

2.2.1. Deliverables

The PneumoAI project is expected to produce a comprehensive set of deliverables that collectively demonstrate the successful design, development, and implementation of an intelligent healthcare application. These deliverables are not limited to a functional system but also include supporting components such as machine learning models, system features, and formal documentation. Each deliverable plays a critical role in ensuring that the project meets both technical and academic requirements.

Firstly, the primary deliverable is a fully functional web-based application designed for the detection of Pneumonia. This application serves as the main interface through which users can interact with the system. It allows users to upload chest X-ray images, receive diagnostic predictions, and access additional features such as history tracking and chatbot interaction. The system is designed to be accessible, responsive, and user-friendly, ensuring usability across different devices and user groups.

A key technical deliverable of the project is the development of a trained deep learning model based on Convolutional Neural Networks (CNNs). This model is responsible for analyzing medical images and

classifying them into relevant categories (e.g., Normal or Pneumonia). The model is trained using labeled datasets and optimized to achieve high accuracy and reliability. Its performance is evaluated using appropriate metrics to ensure that it meets the standards required for a diagnostic support system.

Another important deliverable is the integration of an AI-powered conversational assistant, built using a Large Language Model (LLM). This chatbot enhances the functionality of the system by providing users with contextual information related to pneumonia, including symptoms, preventive measures, and recovery guidelines. It enables natural language interaction, making the system more intuitive and accessible, especially for users without technical or medical expertise.

Security and user management are also critical components of the system. Therefore, a secure user authentication and account management system is included as a core deliverable. This feature ensures that user data is protected and allows personalized functionalities such as saving prediction history and generating reports. Authentication mechanisms such as login and registration are implemented to maintain system integrity and data privacy.

In addition, the system includes a dashboard that visually presents prediction results along with confidence scores. This dashboard is designed to provide clear and interpretable outputs, enabling users to understand the results easily. The inclusion of confidence scores adds transparency to the AI model's predictions, helping users assess the reliability of the output.

The project also delivers a history management feature that stores past predictions and user interactions. This allows users to review previous results, track changes over time, and maintain a record of their diagnostic history. Such functionality is particularly useful for monitoring health trends and supporting long-term analysis.

Another significant deliverable is the PDF report generation feature, which enables users to download and share their diagnostic results in a structured format. This feature enhances the practical usability of the system, as users can easily present these reports to healthcare professionals for further consultation.

Furthermore, an administrative panel is developed to monitor system activity and manage users. This panel provides functionalities such as viewing user data, tracking system usage, and maintaining overall system performance. It ensures that the application can be effectively managed and maintained over time.

Finally, comprehensive project documentation is included as a crucial deliverable. This consists of formal documents such as the Project Proposal, Software Requirements Specification (SRS), Software Design Document (SDD), and the Final Report. These documents provide detailed insights into the system's design, development process, and evaluation, ensuring that the project meets academic standards and can be understood and reviewed effectively.

In summary, the deliverables of the PneumoAI project encompass both technical and documentation components, ensuring a complete and well-structured system. Together, these deliverables demonstrate the successful integration of AI technologies into a practical, user-oriented healthcare application.

2.2.2. Development Requirements

The development of PneumoAI requires a combination of modern web technologies, machine learning frameworks, and supporting tools to ensure the system is scalable, efficient, and reliable. The selection of these technologies is based on their performance, community support, compatibility, and suitability for building an AI-driven web application. Each component of the system architecture is carefully chosen to fulfill specific functional and non-functional requirements.

➢ Frontend Technologies

The frontend of the application is developed using Next.js, a powerful React-based framework that supports server-side rendering and static site generation. This enhances application performance, improves loading speed, and contributes to better user experience. Next.js also provides a structured development environment, making it easier to manage large-scale applications.

To design a responsive and visually consistent user interface, Tailwind CSS is used. Tailwind CSS enables rapid UI development through utility classes, reducing the need for custom styling while maintaining design flexibility. Additionally, ShadCN UI is utilized to provide pre-built, accessible, and customizable UI components, further improving development efficiency and interface consistency.

➢ Backend Technologies

The backend of PneumoAI is implemented using Node.js and Express.js. Node.js enables the use of JavaScript on the server side, allowing seamless integration with the frontend. Express.js provides a lightweight and flexible framework for building RESTful APIs, handling HTTP requests, and managing server-side logic. This combination ensures efficient communication between the frontend, database, and AI services.

➢ Database Management

For data storage, PostgreSQL is selected as the primary database management system. PostgreSQL is a robust and reliable relational database that supports complex queries, data integrity, and scalability. It is used to store structured data such as user information, prediction history, chat logs, and system records. The use of a relational database ensures consistency and efficient data retrieval.

➢ Machine Learning Frameworks

The AI component of the system is developed using Python, along with deep learning frameworks such as TensorFlow and PyTorch. These frameworks provide powerful tools for building, training, and evaluating models based on Convolutional Neural Networks (CNNs). They offer flexibility, scalability, and extensive community support, making them suitable for medical image analysis tasks.

➢ Model Deployment

To deploy the trained model, lightweight web frameworks such as Flask or FastAPI are used. These frameworks enable the creation of a microservice architecture, where the AI model operates as an independent service. This approach improves scalability, allows separate updates to the model, and ensures efficient handling of prediction requests without affecting the main application.

➢ Image Storage

Since the system involves uploading and processing medical images, a cloud-based storage solution such as Cloudinary is utilized. This ensures secure, scalable, and efficient storage of image data. Cloud storage also enables fast retrieval and reduces the load on the main server.

➢ AI Integration

For the conversational AI component, external APIs based on Large Language Models are integrated. Services such as OpenAI or Hugging Face provide pre-trained models capable of understanding and generating natural language. These APIs enable the chatbot to deliver context-aware responses, enhancing user interaction and support.

➢ Version Control and Collaboration

Version control is managed using Git and GitHub. These tools facilitate collaborative development, track changes in the codebase, and ensure proper version management. They also support branching and merging strategies, which are essential for managing different features and updates during development.

➢ Development Environment

The development process is supported by tools such as Visual Studio Code and Jupyter Notebook. Visual Studio Code provides a versatile environment for coding, debugging, and managing project files, while Jupyter Notebook is used for developing and experimenting with machine learning models. These tools enhance productivity and streamline the development workflow.

In conclusion, the development requirements of PneumoAI reflect a carefully selected technology stack that supports both web application development and AI integration. The combination of these tools ensures that the system is scalable, maintainable, and capable of delivering reliable performance in a real-world healthcare context.

2.3. Current System

The current approach to detecting Pneumonia is primarily based on traditional healthcare systems that rely on clinical examination and the manual interpretation of chest X-ray images by trained radiologists. This method has been widely used for decades and remains a standard practice in medical diagnosis. When performed by experienced professionals, it can yield accurate and reliable results. However, despite its effectiveness, this approach is associated with several significant limitations that affect its efficiency, accessibility, and scalability in modern healthcare environments.

One of the most critical challenges of the current system is limited accessibility. In many regions, particularly in rural and underdeveloped areas, there is a shortage of qualified radiologists and diagnostic facilities. This lack of expertise creates barriers to timely diagnosis, as patients may need to travel long distances or wait extended periods to receive medical attention. Consequently, delays in diagnosis can lead to disease progression and increased risk of complications.

Another major limitation is the time-consuming nature of the diagnostic process. Radiologists must carefully analyze each X-ray image, identify abnormalities, and make clinical judgments based on their expertise. In high-demand environments, such as hospitals with large patient volumes, this process can become slow and inefficient. Delayed diagnosis not only affects patient outcomes but also places additional pressure on healthcare systems.

Human error is also an inherent concern in manual image interpretation. Although radiologists are highly trained, factors such as fatigue, workload, and subjective judgment can influence diagnostic decisions. Variability in interpretation between different professionals may result in inconsistent outcomes, including false positives or false negatives. Such errors can have serious consequences, especially in critical conditions like pneumonia, where early detection is essential.

In addition to these limitations, current systems often lack integration and user-centered design. Most traditional diagnostic processes focus solely on identifying the disease and do not provide additional support to patients. There is typically no integrated mechanism for offering educational guidance, explaining results in simple terms, or assisting users in understanding their condition. This creates a gap between diagnosis and patient awareness, leaving individuals dependent on external sources for information.

Cost is another significant factor affecting the accessibility of advanced diagnostic solutions. While modern healthcare facilities may utilize sophisticated imaging equipment and AI-assisted tools, these technologies are often expensive and require specialized infrastructure. As a result, small clinics, independent practitioners, and individuals may not have access to such resources, limiting the widespread adoption of advanced diagnostic systems.

In recent years, AI-based solutions have emerged as potential alternatives to traditional methods. For example, platforms such as Qure.ai offer automated radiology solutions that can analyze medical images efficiently. Similarly, research models like CheXNet, developed using Deep Learning techniques, demonstrate high accuracy in detecting pneumonia from chest X-rays. These advancements highlight the potential of AI in transforming medical diagnosis.

However, despite their capabilities, these solutions also have limitations. Commercial systems often require significant financial investment and are primarily designed for large healthcare institutions. On the other hand, research-based models are typically not integrated into user-friendly platforms and lack features necessary for real-world deployment, such as user interfaces, data management, and interactive support.

Overall, the current system for pneumonia detection, while effective in controlled environments, faces challenges related to accessibility, efficiency, cost, and integration. These limitations highlight the need for a more comprehensive and accessible solution that combines accurate diagnostic capabilities with user-friendly features and educational support. The PneumoAI system is proposed to address these shortcomings by providing an integrated, AI-driven platform that enhances both diagnosis and user interaction.

2.4. Proposed System

The proposed system, PneumoAI, is designed as an integrated, intelligent, and user-centric solution aimed at addressing the limitations identified in existing pneumonia detection approaches. By leveraging advancements in Artificial Intelligence and modern web technologies, the system combines automated disease detection with interactive user support within a single, cohesive platform. The primary objective of PneumoAI is to enhance accessibility, improve diagnostic efficiency, and provide meaningful guidance to users, particularly in environments where medical resources are limited.

At the core of the proposed system is an automated diagnostic component based on Deep Learning, specifically utilizing Convolutional Neural Networks (CNNs). This model is trained on chest X-ray images to identify patterns associated with Pneumonia and classify them into relevant categories. The use of CNNs enables the system to perform image analysis with high accuracy and consistency, reducing reliance on manual interpretation. This automated detection mechanism forms the foundation of the system's functionality.

One of the key features of PneumoAI is the provision of real-time prediction results accompanied by confidence scores. These scores represent the model's certainty in its predictions, allowing users to better interpret the output. This transparency is essential in AI-driven systems, particularly in healthcare applications, where users must understand the reliability of the results before making decisions.

In addition to diagnostic capabilities, the system integrates an AI-powered conversational assistant based on a Large Language Model (LLM). This chatbot enables users to interact with the system using natural language, providing information related to pneumonia, including symptoms, causes, preventive measures, and recovery practices. By combining diagnosis with educational support, the system enhances user engagement and helps bridge the gap between technical outputs and user understanding.

The proposed system is implemented as a secure and user-friendly web application, ensuring accessibility across different devices and user groups. The interface is designed with a focus on simplicity and usability, allowing users to upload images, view results, and interact with the chatbot without requiring specialized knowledge. Security measures, including authentication and data protection mechanisms, are incorporated to safeguard user information.

Another important feature of PneumoAI is the storage of user history and prediction records. This functionality enables users to track their past results, monitor changes over time, and maintain a record of their diagnostic data. Such historical tracking can be valuable for both users and healthcare professionals in assessing health trends and making informed decisions.

To further enhance usability, the system includes a PDF report generation feature. This allows users to download their diagnostic results in a structured and shareable format, which can be presented to medical professionals for further consultation. This feature bridges the gap between digital diagnosis and traditional healthcare practices.

Additionally, an administrative panel is integrated into the system to support monitoring and management activities. This panel provides administrators with tools to oversee system performance, manage user accounts, and ensure the smooth operation of the application. It also facilitates maintenance and scalability as the system evolves.

The overall design of PneumoAI emphasizes scalability, cost-effectiveness, and accessibility. By utilizing a modular architecture and cloud-based services, the system can be easily scaled to accommodate a growing number of users. Furthermore, the use of open-source technologies and efficient deployment strategies helps reduce development and operational costs, making the system more accessible to a wider audience.

It is important to note that PneumoAI is not intended to replace professional medical diagnosis. Instead, it is designed as a supportive tool that assists in early detection and increases awareness. By providing preliminary insights and educational guidance, the system empowers users to take informed actions while encouraging consultation with healthcare professionals.

In conclusion, the proposed system offers a comprehensive solution that integrates automated diagnosis, interactive support, and user-friendly design. By addressing the limitations of existing systems, PneumoAI represents a practical and innovative approach to enhancing pneumonia detection and healthcare accessibility.

CHAPTER # 3

3. Requirement Analysis

This chapter presents a comprehensive analysis of the requirements for the PneumoAI system, forming a critical foundation for its design and implementation. Requirement analysis is a fundamental phase in Software Engineering, as it defines what the system is expected to do and how it should perform under various conditions. A clear and well-structured requirement specification ensures that the final system meets user needs, aligns with project objectives, and minimizes the risk of design inconsistencies or implementation errors.

The purpose of this chapter is to identify, organize, and document both functional and non-functional requirements of the system. Functional requirements describe the core operations and features that the system must provide, such as user authentication, image upload, pneumonia detection, and chatbot interaction. These requirements define the system's behavior and outline how users will interact with different components. On the other hand, non-functional requirements focus on the quality attributes of the system, including performance, scalability, security, usability, and reliability. Together, these requirements ensure that the system is not only functional but also efficient and user-friendly.

To provide a structured representation of system behavior, this chapter also incorporates modeling techniques such as use case diagrams and detailed use case descriptions. These models visually illustrate the interactions between users and the system, helping to clarify system functionality and identify key processes. Use case diagrams, in particular, play an important role in capturing system requirements from a user's perspective, highlighting the roles of different actors and their interactions with the system.

In addition, this chapter emphasizes the importance of understanding user needs and system constraints. PneumoAI is designed to be accessible to both healthcare professionals and general users, which requires careful consideration of usability and accessibility factors. The system must be intuitive, responsive, and capable of handling various user scenarios, including image uploads, result interpretation, and interaction with the AI assistant.

Another key aspect of requirement analysis is ensuring that the system aligns with real-world challenges identified in previous chapters. The requirements defined in this chapter are directly influenced by the limitations of existing systems, such as lack of accessibility, limited integration, and absence of user-centered features. By addressing these gaps, the requirement analysis ensures that the proposed system delivers a comprehensive and practical solution.

Furthermore, this chapter serves as a bridge between conceptual understanding and technical implementation. The requirements outlined here will guide the design decisions in subsequent chapters, including system architecture, database design, and component integration. A well-defined requirement analysis not only simplifies the development process but also ensures that the final system meets both functional expectations and quality standards.

In summary, this chapter provides a detailed and structured specification of the PneumoAI system requirements. It lays the groundwork for system design by clearly defining expected behaviors, user interactions, and performance criteria. Through the use of diagrams and detailed descriptions, it ensures that all aspects of the system are well understood before moving into the design and implementation phases.

3.1. Use Case Diagram

The use case diagram presented above provides a high-level representation of the interactions between users and the PneumoAI system. It illustrates how different actors engage with the system's functionalities and defines the overall flow of operations. The diagram focuses on two primary actors: the **User (Patient)** and the **Admin**, each with distinct roles and responsibilities within the system.

The **User (Patient)** is the main actor who interacts with the system to perform core functionalities related to pneumonia detection and health assistance. The process begins with account creation through the *Sign Up* feature, followed by *Login* to access the system securely. Once authenticated, the user can utilize the primary functionality of the system, which is *Pneumonia Prediction*. This involves uploading a chest X-ray image, which is then processed by the system to generate a diagnostic result.

After the prediction is completed, the user is able to view the results along with associated confidence levels, helping them understand the likelihood of the diagnosis. In addition to prediction, the user can interact with the system through the *Ask Health Related Question to AI* functionality. This feature enables communication with the AI assistant, allowing users to receive guidance regarding pneumonia-related concerns such as symptoms, prevention, and recovery.

The use case diagram also reflects that both *Login* and *Sign Up* are essential entry points for user interaction, ensuring secure access and personalized system usage. Although not explicitly shown as separate ovals in the diagram, functionalities such as viewing history and downloading reports are conceptually part of the user's interaction flow after prediction.

On the other hand, the **Admin** actor is responsible for managing and monitoring the system. The admin interacts with the system primarily through *Login*, after which they can perform administrative tasks. One of the key functionalities available to the admin is *Monitor Patient Activity*, which allows oversight of user interactions and system usage. Additionally, the admin has the ability to *Moderate AI Queries*, ensuring that the responses generated by the AI assistant remain appropriate, accurate, and aligned with system objectives.

The diagram clearly shows that administrative functionalities are separate from user operations, ensuring a structured and secure system design. The admin does not directly interact with prediction features but instead focuses on maintaining system integrity and performance.

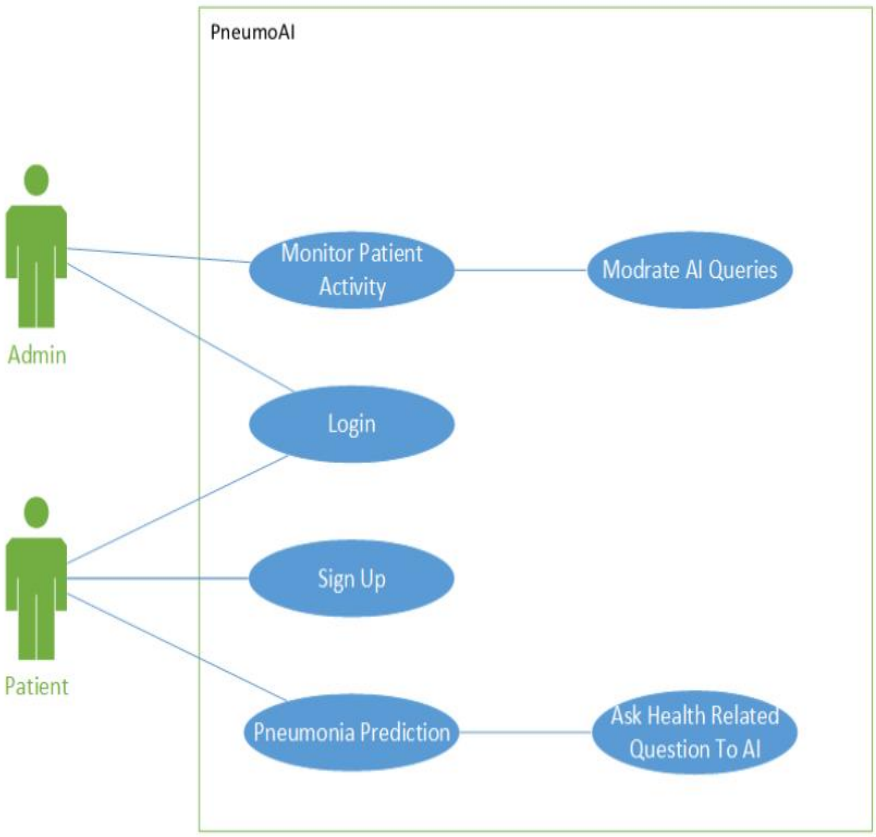

Figure 3.1: Use Case Diagram of PneumoAI System

Overall, this use case diagram represents the complete workflow of PneumoAI, starting from user registration and authentication, progressing through image-based pneumonia prediction, and extending to interactive communication with the AI assistant. It also highlights the administrative controls required for system monitoring and management. By providing a clear visualization of actor interactions, the diagram serves as a foundation for understanding system behavior and guides the subsequent design and implementation phases.

3.2. Detailed Use Case

The following use case provides a detailed description of the primary interaction between the **User (Patient)** and the PneumoAI system. It represents the complete operational workflow, covering all major steps from system access to obtaining prediction results and interacting with the AI assistant. This use case is essential for understanding how different system components collaborate to deliver the intended functionality.

Use Case ID: UC-1

Use Case Name: Complete PneumoAI User Interaction

Actors: User (Patient)

Description:

This use case describes the complete workflow of a user interacting with the PneumoAI system, starting from account creation and authentication to uploading a chest X-ray image, receiving a pneumonia prediction, and interacting with the AI assistant for further guidance. It reflects the core functionality of the system and demonstrates how multiple components such as authentication, image processing, deep learning models, and conversational AI are integrated into a seamless user experience.

Preconditions:

- The user must have access to a device (e.g., computer or smartphone) with a stable internet connection.
- The user must be registered in the system to access login functionality.

Postconditions:

- The prediction result (Normal or Pneumonia) is displayed along with a confidence score.
- The prediction data is securely stored in the database for future reference.
- The user is able to download a report or interact with the AI assistant for additional guidance.

Normal Flow:

1. The user accesses the PneumoAI web application through a browser.
2. The user either signs up (if new) or logs in using existing credentials.
3. The system authenticates the user credentials and grants access to the dashboard.
4. The user uploads a chest X-ray image through the provided interface.
5. The system validates the uploaded file based on format, size, and compatibility.
6. The validated image is forwarded to the deep learning model based on Convolutional Neural Networks for analysis.
7. The model processes the image and returns a prediction (Normal or Pneumonia) along with a confidence score.
8. The system stores the prediction result and associated metadata in the database.
9. The result is displayed to the user on the dashboard in a clear and interpretable format.
10. The user has the option to download the result as a PDF report for further use.
11. The user may interact with the AI assistant to ask health-related questions regarding Pneumonia.
12. The system processes the query and returns a response generated by the AI assistant.

Alternative Flows:

- If the user enters invalid login credentials, the system displays an error message and prompts for correction.
- If the uploaded file does not meet the required format or size constraints, the system rejects the upload and provides feedback.
- If the prediction service (AI model) is unavailable, the system displays a fallback message and may suggest retrying later.
- If the AI assistant service is unavailable, the system provides a retry option or informs the user accordingly.

Exceptions:

- Network failure during image upload may interrupt the process, requiring the user to reattempt submission.
- Server-side errors during processing may prevent the system from generating results.
- Timeout issues in AI or machine learning services may delay or fail the response generation.

Analytical Insight:

This use case highlights the integration of multiple subsystems within PneumoAI, including authentication, image processing, AI-based prediction, data storage, and conversational interaction. It demonstrates how the system ensures a smooth and continuous workflow while handling potential errors and alternative scenarios. By covering both normal and exceptional conditions, this use case provides a complete understanding of system behavior and serves as a reference for system design, testing, and validation.

3.3. Functional Requirements

The functional requirements define the core operations and system behaviors that PneumoAI must perform to achieve its objectives. These requirements describe how users and administrators interact with the system and how different components such as authentication, image processing, AI prediction, and data management work together to deliver the intended functionality. Each requirement is uniquely identified and specifies a particular feature of the system.

➢ FR-1: User Registration

The system shall allow users to create an account using an email address and password.

- The registration process must include input validation (e.g., valid email format, password strength).
- The system shall prevent duplicate account creation using the same email.
- Upon successful registration, user data shall be securely stored in the database.

➢ FR-2: User Login

The system shall authenticate users and allow secure login.

- Users must provide valid credentials (email and password).
- The system shall verify credentials against stored records.
- Upon successful authentication, users shall be granted access to the system dashboard.
- Invalid login attempts shall trigger appropriate error messages.

➢ FR-3: Image Upload

The system shall allow users to upload chest X-ray images for analysis.

- Supported file formats shall include JPG, PNG, and DICOM.
- The upload interface shall be simple and user-friendly.
- The system shall ensure secure transfer of image data to the server.

➢ FR-4: Image Validation

The system shall validate uploaded images before processing.

- The system shall check file type and size constraints.
- Invalid or unsupported files shall be rejected with an appropriate message.
- Only validated images shall proceed to the prediction stage.

➢ FR-5: Prediction Processing

The system shall process uploaded images using a deep learning model based on Convolutional Neural Networks.

- The system shall send validated images to the AI model service.
- The model shall analyze the image and classify it as Normal or Pneumonia.

- The prediction process shall be optimized for performance and accuracy.

➢ FR-6: Result Display

The system shall display prediction results to the user.

- Results shall include classification (Normal or Pneumonia) and a confidence score.
- The output shall be presented in a clear and understandable format.
- Visual elements (e.g., indicators or charts) may be used to enhance interpretation.

➢ FR-7: History Management

The system shall store and display previous prediction records.

- Each record shall include image details, prediction result, and timestamp.
- Users shall be able to view their history through a dedicated dashboard section.
- The system shall ensure secure storage and retrieval of historical data.

➢ FR-8: Report Generation

The system shall allow users to download prediction results in PDF format.

- The report shall include user details, prediction outcome, and confidence score.
- Reports shall be formatted in a structured and professional layout.
- Users shall be able to download or share reports.

➢ FR-9: AI Health Assistant

The system shall provide a chatbot powered by a Large Language Model.

- The chatbot shall allow users to ask questions related to Pneumonia.
- It shall provide information on symptoms, prevention, and recovery.
- The interaction shall be conversational and user-friendly.

➢ FR-10: Context-Aware Responses

The chatbot shall provide responses based on user context and system data.

- Responses may consider prediction results and user queries.
- The system shall ensure that responses are relevant and informative.
- Context-awareness shall enhance the usefulness of the chatbot.

➢ FR-11: Admin Monitoring

The system shall allow administrators to monitor system activity.

- Admins shall be able to view user interactions and system usage statistics.
- The system shall provide logs for tracking important events.
- Monitoring tools shall support system maintenance and performance evaluation.

➢ FR-12: User Management

The system shall allow administrators to manage user accounts.

- Admins shall be able to view user profiles and activity.
- The system shall allow restricting or disabling accounts if necessary.
- User management shall ensure system security and proper usage control.

Summary of Functional Scope

These functional requirements collectively define the operational capabilities of PneumoAI. They ensure that the system supports secure user interaction, accurate AI-based prediction, efficient data management, and meaningful user guidance. By clearly specifying each function, this section provides a solid foundation for system design, implementation, and testing.

3.4. Non-Functional Requirements

Non-functional requirements define the quality attributes, constraints, and performance expectations of the PneumoAI system. While functional requirements describe what the system does, non-functional requirements specify how well the system performs those functions. These requirements are critical for ensuring that the system is efficient, secure, reliable, and scalable in real-world environments.

1. Usability Requirements

Usability focuses on ensuring that the system is easy to learn, intuitive to use, and accessible to a wide

range of users, including individuals without technical expertise.

- The system shall provide an intuitive and user-friendly interface with clear navigation.
- Users shall be able to upload images and receive results within minimal steps, reducing complexity in interaction.
- The dashboard shall present prediction results clearly, using visual indicators such as labels, icons, or progress bars to enhance understanding.
- The interface shall be responsive and compatible across different devices, including desktops, tablets, and mobile devices.
- Instructions and feedback messages shall be clear, concise, and easy to interpret.

2. Performance Requirements

Performance requirements ensure that the system operates efficiently and delivers results within acceptable time limits.

- The system shall load web pages within 3 seconds under normal operating conditions.
- Prediction results generated using the Convolutional Neural Networks model shall be delivered within 5 seconds.
- Responses from the AI assistant, powered by a Large Language Model, shall be generated within 3 seconds.
- The system shall support at least 50 concurrent users without significant performance degradation.
- Backend services shall be optimized to handle multiple requests efficiently through asynchronous processing where applicable.

3. Security Requirements

Security is a critical requirement, particularly for systems handling sensitive user data and medical information.

- All data transmitted between the client and server shall be encrypted using HTTPS protocols.
- User passwords shall be securely stored using hashing techniques such as Bcrypt to prevent unauthorized access.
- Authentication shall be managed using secure token-based mechanisms (e.g., JWT) to ensure session integrity.
- Access to administrative features shall be restricted to authorized users only through role-based access control.

- The system shall protect against common vulnerabilities such as SQL injection, cross-site scripting (XSS), and unauthorized data access.

4. Reliability Requirements

Reliability ensures that the system performs consistently under defined conditions and handles failures effectively.

- The system shall handle errors gracefully by displaying meaningful fallback messages to users.
- Failures in prediction services or AI components shall not cause the entire system to crash.
- The system shall implement logging mechanisms to track errors and system behavior.
- Data shall be consistently stored and retrieved without loss or corruption.
- Backup mechanisms may be implemented to ensure data recovery in case of failure.

5. Scalability Requirements

Scalability defines the system's ability to handle growth in users, data, and processing demands.

- The system shall support an increasing number of users and prediction requests without performance degradation.
- A microservices architecture shall be used to allow independent scaling of components such as the machine learning service and backend server.
- Cloud-based infrastructure shall enable dynamic resource allocation based on system demand.
- The database shall be optimized to handle large volumes of structured data efficiently.

6. Availability Requirements

Availability ensures that the system remains accessible and operational for users at all times.

- The system shall be available 24/7 with minimal downtime.
- Cloud hosting services shall be utilized to ensure high availability and redundancy.
- Mechanisms such as load balancing and failover strategies may be implemented to maintain system uptime.
- Scheduled maintenance activities shall be performed with minimal disruption to users.

7. Maintainability Requirements

Maintainability focuses on the ease with which the system can be updated, modified, and extended over time.

- The system shall follow a modular design approach, allowing individual components to be updated independently.
- The codebase shall be well-structured, readable, and documented to support future development.
- Standard coding practices and version control systems shall be used to manage changes effectively.
- The architecture shall support the addition of new features, such as enhanced AI models or additional reporting functionalities, without major system redesign.

Summary of Quality Attributes

The non-functional requirements outlined above ensure that PneumoAI is not only functionally complete but also efficient, secure, reliable, and scalable. These quality attributes are essential for delivering a robust system that can operate effectively in real-world healthcare scenarios while providing a seamless user experience.

CHAPTER # 4

4. Design and Architecture

This chapter presents a comprehensive description of the design and architecture of the PneumoAI system, outlining how its various components are structured and how they interact to deliver the intended functionality. System design is a critical phase in Software Engineering, as it translates the requirements identified in the previous chapter into a concrete technical blueprint. A well-defined architecture ensures that the system is scalable, maintainable, and capable of handling real-world operational demands.

The PneumoAI system follows a modular and layered architecture, combining modern web development practices with AI-driven services. The system is composed of three primary layers: the frontend (user interface), the backend (application logic), and the AI/ML service (prediction engine). These layers are interconnected through well-defined interfaces and communication protocols, ensuring smooth data flow and separation of concerns.

The **frontend layer** is responsible for user interaction and presentation. It provides an intuitive interface through which users can perform actions such as registration, login, image upload, viewing prediction results, and interacting with the AI assistant. The design emphasizes usability and responsiveness, ensuring that users can easily navigate the system across different devices. This layer communicates with the backend through API requests, sending user inputs and receiving processed results.

The **backend layer** acts as the central controller of the system. It handles business logic, user authentication, request processing, and communication between different components. When a user uploads an image, the backend validates the input and forwards it to the AI service for analysis. Once the prediction is received, the backend stores the result in the database and sends it back to the frontend for display. The backend also manages user sessions, history records, and administrative functionalities.

A key component of the architecture is the **AI/ML service**, which is responsible for performing pneumonia detection. This service is implemented as a separate microservice using frameworks such as Flask or FastAPI, allowing it to operate independently of the main application. The model, based on Convolutional Neural Networks, processes chest X-ray images and returns classification results. By isolating the AI component, the system achieves better scalability and flexibility, as the model can be updated or replaced without affecting other parts of the application.

The system also incorporates a conversational AI component powered by a Large Language Model. This component enables natural language interaction between users and the system, providing contextual guidance related to Pneumonia. The chatbot operates through API integration and enhances the overall user experience by making the system more interactive and informative.

Data management is handled through a relational database system, which stores user information, prediction history, and system logs. The database is designed to ensure data integrity, efficient retrieval, and secure storage. Additionally, cloud-based storage solutions are used for handling uploaded images, reducing the load on the main server and improving scalability.

The architecture also supports a microservices-based approach, where different components such as the backend server, AI model, and chatbot service operate as independent services. This design enables better

fault isolation, scalability, and maintainability. For example, if the AI service experiences issues, the rest of the system can continue functioning without complete failure.

Furthermore, the system design incorporates secure communication protocols, ensuring that data exchanged between components is protected. Authentication mechanisms and access controls are implemented to safeguard user information and restrict unauthorized access.

From a behavioral perspective, the system follows a structured data flow: user input is captured through the frontend, processed by the backend, analyzed by the AI service, and then returned to the user in an interpretable format. This flow ensures that each component performs a specific role, contributing to the overall efficiency and reliability of the system.

In conclusion, the design and architecture of PneumoAI provide a robust and scalable framework that integrates web technologies with AI capabilities. By adopting a modular and microservices-based approach, the system ensures flexibility, maintainability, and efficient performance. This architectural foundation supports the implementation of complex functionalities while maintaining a clear separation of concerns, ultimately contributing to the effectiveness of the proposed solution.

4.1. System Architecture

The PneumoAI system is designed using a microservices-based layered architecture, which ensures modularity, scalability, and efficient integration of intelligent components. This architectural approach separates the system into distinct layers, each responsible for specific functionalities, thereby improving maintainability and flexibility. By adopting this design, the system can evolve over time, allowing independent updates to individual components without affecting the entire application.

The architecture is composed of five primary layers: Presentation Layer, Application Layer, Machine Learning Service, AI Assistant Service, and Data Layer. Each layer communicates through well-defined interfaces, ensuring smooth data flow and clear separation of concerns.

1. Presentation Layer (Frontend)

The presentation layer represents the user-facing component of the system. It is developed using Next.js and Tailwind CSS, which together provide a responsive, efficient, and visually consistent user interface. This layer is responsible for handling all user interactions, including registration, login, image upload, result visualization, and communication with the AI assistant.

The frontend communicates with the backend through API calls, sending user inputs such as images and queries, and receiving processed results for display. The design prioritizes usability and accessibility, ensuring that users can navigate the system and perform tasks with minimal complexity.

2. Application Layer (Backend API)

The application layer acts as the central control unit of the system and is implemented using Node.js and

Express.js. This layer is responsible for managing the core logic of the application and coordinating communication between different services.

Key responsibilities of this layer include:

- User authentication using secure token-based mechanisms (JWT)
- Handling API requests and routing them to appropriate services
- Validating input data before processing
- Communicating with the machine learning service for predictions
- Integrating with the AI assistant service for chatbot functionality

By centralizing these operations, the backend ensures consistency, security, and efficient processing of user requests.

3. Machine Learning Service (ML Microservice)

The machine learning component is implemented as a separate microservice using Flask. This design allows the AI model to operate independently of the main application, improving scalability and flexibility.

This service performs the following tasks:

- Receives chest X-ray images from the backend
- Applies preprocessing techniques such as resizing and normalization
- Executes model inference using a Convolutional Neural Networks
- Returns classification results (Normal or Pneumonia) along with confidence scores

By isolating the ML service, updates to the model (e.g., retraining or optimization) can be performed without disrupting the rest of the system.

4. AI Assistant Service (LLM Integration)

The system integrates a conversational AI component powered by a Large Language Model through an external API. This service enables natural language interaction, allowing users to ask questions related to Pneumonia.

The backend constructs controlled and context-aware prompts before sending them to the LLM API. This ensures that responses remain relevant, accurate, and aligned with the system’s purpose. The AI assistant enhances the system by providing educational support, making the application more interactive and user-friendly.

5. Data Layer

The data layer is responsible for storing and managing all system data. PneumoAI uses a hybrid storage approach to handle different types of data efficiently:

- PostgreSQL is used for storing structured data such as user accounts, prediction history, and chatbot interactions.
- Cloudinary or similar cloud-based solutions are used for storing uploaded chest X-ray images.

This separation ensures efficient data handling, improved performance, and scalability.

Architecture Characteristics

The chosen architecture provides several key advantages:

- **Modular Design:** Each component operates independently, allowing easier development, testing, and maintenance.
- **Scalability:** Individual services, such as the ML model or backend API, can be scaled independently based on demand.
- **Security:** Secure communication protocols (HTTPS) and authentication mechanisms ensure data protection.
- **Maintainability:** Clear separation of concerns simplifies updates, debugging, and future enhancements.

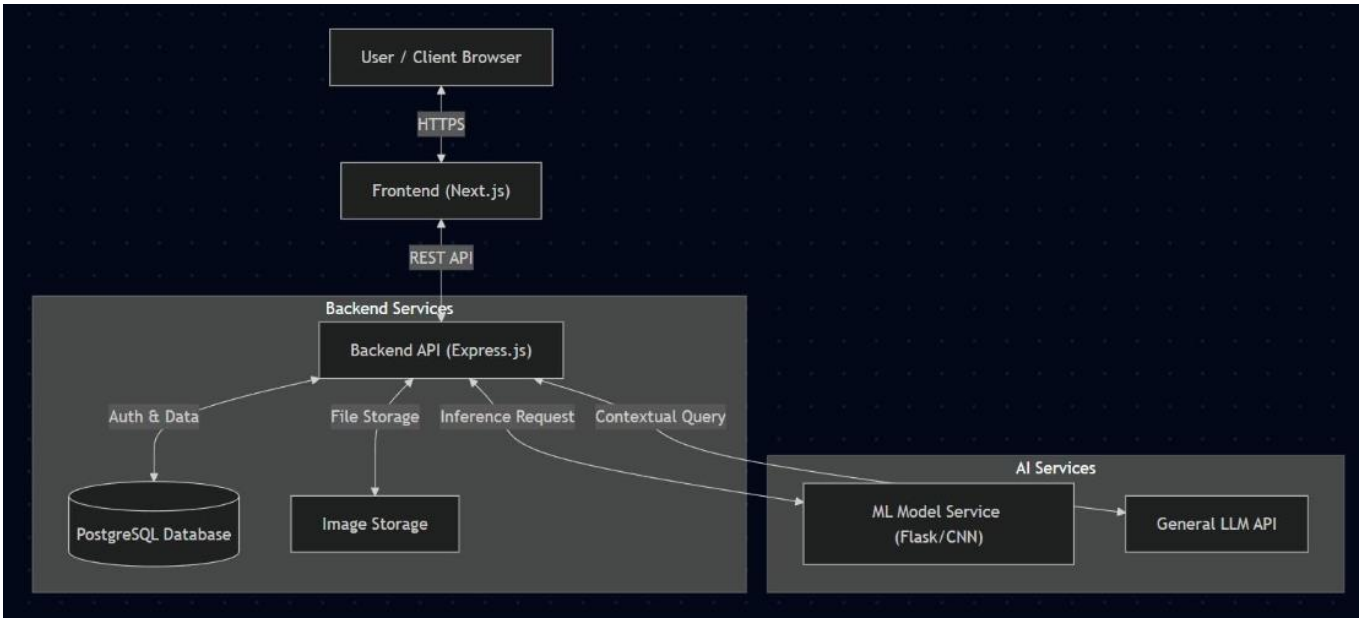

Figure 4.1: System Architecture of PneumoAI

Overall System Flow

From a system perspective, the workflow follows a structured sequence:

User input is captured through the frontend → processed by the backend → forwarded to the ML or AI service → results are stored in the database → and finally displayed to the user. This layered interaction ensures efficient processing, reliability, and a seamless user experience.

4.2. Data Representation

The PneumoAI system organizes and manages data through a structured representation model that ensures efficient storage, retrieval, and processing. Effective data representation is a fundamental aspect of system design in Software Engineering, as it directly impacts system performance, scalability, and maintainability. By defining clear entities and relationships, the system ensures consistency and integrity of data across all components.

Data Entities

The system is built around several core entities that represent different types of data generated and used within the application. These entities are designed to capture both user-related information and system-generated outputs.

- **User:** This entity stores essential user information, including credentials (email and password), profile details, and authentication-related data. It serves as the primary entity for identifying and managing system users.
- **Prediction:** This entity stores information related to pneumonia detection results. It includes attributes such as the image URL, classification result (Normal or Pneumonia), confidence score, timestamp, and processing status. This entity plays a central role in linking user activity with AI-generated outputs.
- **Chat Message:** This entity records interactions between the user and the AI assistant. It includes user queries and corresponding AI-generated responses, enabling conversational history tracking and contextual interaction.

Data Relationships

The relationships between entities are defined to reflect real-world interactions within the system and to ensure logical data organization.

- A **one-to-many relationship** exists between the User and Prediction entities, meaning that one user can have multiple prediction records over time.
- A **one-to-many relationship** also exists between the Prediction and Chat Message entities, where

each prediction can be associated with multiple chat interactions for contextual guidance.

- Images are not stored directly in the database but are referenced through URLs, which are linked to the Prediction entity.

These relationships are enforced using foreign keys within the database, ensuring referential integrity and preventing data inconsistencies.

Storage Strategy

PneumoAI adopts a hybrid storage strategy to efficiently handle both structured and unstructured data:

- **Structured Data:** All structured data, including user information, prediction records, and chat logs, is stored in PostgreSQL. This ensures strong data integrity, support for relational constraints, and efficient query performance.
- **Unstructured Data (Images):** Chest X-ray images are stored in a cloud-based storage service such as Cloudinary. This approach reduces the load on the database and allows efficient handling of large image files.
- **Linking Mechanism:** The system uses image URLs to connect database records with stored image files. This indirect referencing mechanism ensures efficient storage while maintaining easy access to image data when required.

Data Security

Data security is a critical aspect of the system, particularly due to the sensitive nature of user and medical data. Several measures are implemented to ensure data protection:

- User passwords are stored using secure hashing algorithms (e.g., Bcrypt), preventing exposure of plaintext credentials.
- Data transmitted between system components is encrypted using HTTPS protocols to ensure confidentiality and integrity.
- Role-based access control (RBAC) is implemented to restrict access to sensitive features, particularly administrative functionalities.
- Validation and sanitization mechanisms are applied to user inputs to prevent common security vulnerabilities.

Design Considerations and Benefits

The chosen data representation approach provides several advantages:

- **Efficiency:** Structured storage enables fast querying and retrieval of data.

- **Scalability:** Separation of structured and unstructured data allows the system to handle growth effectively.
- **Maintainability:** Clear entity definitions and relationships simplify database management and future updates.
- **Security:** Proper handling of sensitive data ensures compliance with best practices in secure system design.

Summary

The data representation strategy of PneumoAI ensures that all system data is organized, secure, and efficiently accessible. By combining relational database design with cloud-based storage, the system achieves a balance between performance and scalability. This structured approach supports the overall functionality of the system and provides a solid foundation for data-driven operations.

4.3. Process Flow / Representation

The PneumoAI system follows a structured and sequential process flow to manage user interactions, image processing, and AI-driven responses. The process flow diagram (activity diagram) illustrates how data moves through different components of the system, from user input to final output. This representation is essential for understanding system behavior, identifying decision points, and ensuring smooth coordination between the frontend, backend, and AI services.

The overall workflow is divided into two main processes: the **Prediction Flow** and the **AI Consultation Flow**. Each process is designed to handle specific functionalities while maintaining efficiency, reliability, and user experience.

Prediction Flow

The prediction flow represents the core functionality of PneumoAI, where a user uploads a chest X-ray image and receives a diagnostic result. The process begins when the user logs into the system and accesses the dashboard. After authentication, the user uploads a chest X-ray image through the interface.

Once the image is uploaded, the backend performs validation to ensure that the file meets the required format and size constraints. If the image is invalid, the system immediately returns an error message and prompts the user to re-upload a valid file. This validation step is critical for maintaining system integrity and preventing processing errors.

If the image passes validation, it is stored in cloud storage, and its corresponding URL is generated. This URL is then sent to the machine learning service, where the image is processed using a model based on Convolutional Neural Networks. The model analyzes the image and generates a prediction, classifying it as either Normal or Pneumonia, along with a confidence score indicating the certainty of the prediction.

After processing, the prediction result is returned to the backend, where it is stored in the database along with relevant metadata such as timestamp and image reference. Finally, the result is displayed to the user

on the dashboard in a clear and interpretable format, completing the prediction flow.

AI Consultation Flow

The AI consultation flow extends the system's functionality by enabling user interaction with the AI assistant. After receiving the prediction result, the user has the option to ask health-related questions through the chatbot interface.

When a query is submitted, the backend retrieves relevant context, such as the user's recent prediction result, to ensure that the response is personalized and meaningful. The query, along with this contextual information, is then sent to an external AI service powered by a Large Language Model.

The AI service processes the input and generates a response based on the provided context and predefined constraints. This response is returned to the backend and then displayed to the user. Additionally, the interaction is stored in the database as part of the chat history, allowing future reference and continuity in conversation.

Key Characteristics

The process flow of PneumoAI is designed with several important characteristics that enhance system performance and reliability:

- **Asynchronous Processing:** The system uses asynchronous communication between components, allowing multiple operations to occur simultaneously without blocking user interaction. This improves responsiveness and efficiency.
- **Error Handling at Each Stage:** Validation checks and fallback mechanisms are implemented at critical points in the workflow. For example, invalid inputs, service failures, or network issues are handled gracefully with appropriate user feedback.
- **Efficient API Communication:** The frontend, backend, machine learning service, and AI assistant communicate through well-defined APIs. This ensures smooth data exchange and modular interaction between components.
- **Decision-Based Flow Control:** The system includes decision points (e.g., image validation, user query interaction) that guide the flow of operations based on conditions, ensuring logical and controlled execution.

Summary

The process flow representation of PneumoAI provides a clear and structured view of how the system operates internally. By separating the prediction and consultation processes, the system ensures efficient handling of both diagnostic and interactive functionalities. This design not only improves system performance but also enhances user experience by delivering fast, accurate, and context-aware results.

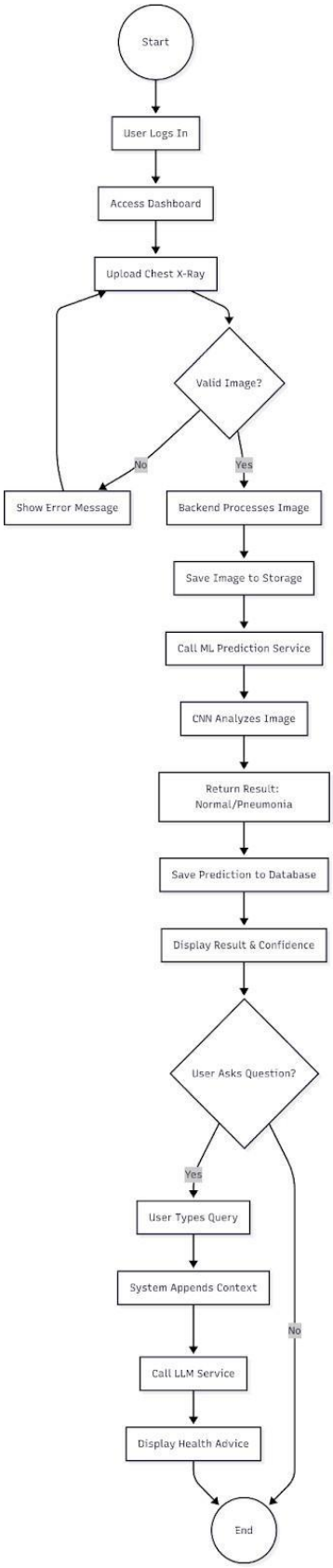

Figure 4.2: Process Flow of PneumoAI System

4.4. Design Models

Design models provide a graphical and conceptual representation of the system's structure and behavior, enabling a clear understanding of how different components are organized and how they interact during execution. In the context of PneumoAI, these models play a crucial role in translating system requirements into a well-defined technical blueprint.

In Software Engineering, design models are used to bridge the gap between requirement analysis and implementation. They help developers, stakeholders, and evaluators visualize the internal workings of the system, ensuring that all components are properly structured and aligned with the intended functionality. By using standardized modeling techniques such as UML (Unified Modeling Language), the system design becomes more systematic, consistent, and easier to interpret.

The design models in PneumoAI focus on both **static structure** and **dynamic behavior**:

- **Structural Models (e.g., Class Diagram):** These models represent the static aspects of the system, including classes, attributes, methods, and relationships. They define how data is organized and how different entities are connected within the system.
- **Behavioral Models (e.g., Sequence Diagram, Activity Diagram):** These models illustrate how the system behaves during execution. They show the flow of operations, interactions between components, and the sequence of events triggered by user actions.

The use of design models provides several important benefits:

- **Clarity:** Visual representations make complex system structures easier to understand.
- **Consistency:** Ensures that all components follow a unified design approach.
- **Error Reduction:** Helps identify design flaws or missing components before implementation.
- **Communication:** Facilitates better understanding among developers, stakeholders, and evaluators.

In PneumoAI, design models are particularly important due to the integration of multiple components such as the frontend interface, backend services, machine learning model, and AI assistant. These models help illustrate how these components interact within a microservices-based architecture and ensure that the system remains modular and scalable.

Overall, the design models provide a comprehensive visualization of the PneumoAI system, supporting both structural organization and dynamic interaction. They serve as a foundation for implementation and play a key role in ensuring that the system is robust, efficient, and aligned with its requirements.

4.4.1. Class Diagram

The class diagram represents the static structure of the PneumoAI system by illustrating the key classes, their attributes, methods, and the relationships between them. It provides a clear view of how data is organized and how different components of the system interact at a structural level. This diagram is an essential part of system modeling in Software Engineering, as it supports object-oriented design and helps

in translating system requirements into implementation-ready structures.

Key Classes:

User

The *User* class represents system users and manages authentication and profile-related data.

Attributes include:

- id (UUID)
- email (String)
- passwordHash (String)
- name (String)
- role (String)
- isBanned (Boolean)

Methods include:

- register()
- login()
- getHistory()

This class is responsible for managing user identity, authentication, and access to system features.

Prediction

The *Prediction* class stores the results generated from analyzing chest X-ray images.

Attributes include:

- id (UUID)
- userId (UUID)
- imageUrl (String)
- result (String)
- confidence (Float)
- status (String)
- createdAt (DateTime)

Methods include:

- create()

- delete()

This class maintains the diagnostic output of the system, including classification results (Normal or Pneumonia) and associated metadata.

Chat Message

The *ChatMessage* class handles communication between the user and the AI assistant.

Attributes include:

- id (UUID)
- userId (UUID)
- predictionId (UUID)
- userMessage (String)
- aiResponse (String)
- timestamp (DateTime)

Methods include:

- saveMessage()

This class ensures that all interactions with the AI assistant are recorded and can be retrieved for context-aware communication.

App Server Class

The *AppServer* class represents the backend application layer that orchestrates system operations.

Methods include:

- handleUpload(file)
- orchestratePrediction()
- validateAuth()

This class acts as the central controller, managing communication between the frontend, machine learning service, and AI assistant service.

ML Service Class

The *MLService* class represents the machine learning microservice responsible for image analysis.

Attributes include:

- modelPath (String)

Methods include:

- preprocessImage(image)
- runInference(tensor)

This service uses a model based on Convolutional Neural Networks to analyze X-ray images and generate prediction results.

External LLM Service Class

The *External LLM Service* class represents the integration with an external AI assistant service. **Attributes include:**

- apiKey (String)

Methods include:

- generateHealthAdvice(context, query): String

This service leverages a Large Language Model to provide context-aware responses to user queries related to pneumonia.

Class Relationships

The relationships between the classes define how different components of the system interact:

- **User → Prediction (One-to-Many):** A single user can have multiple prediction records. This relationship is represented through the userId foreign key in the Prediction class.
- **Prediction → Chat Message (One-to-Many):** Each prediction can be associated with multiple chat messages, allowing users to ask follow-up questions based on specific results.
- **User → Prediction (Management Relationship):** The user manages their prediction records, including viewing and retrieving history.
- **Prediction → Chat Message (Contextual Relationship):** Chat messages are linked to predictions to maintain contextual relevance in AI interactions.
- **App Server → ML Service / External LLM Service (Dependency Relationship):** The App Server uses both the ML Service and External LLM Service to process predictions and generate AI responses, reflecting a service-oriented architecture.

Object-Oriented Design Principles

The class diagram demonstrates key object-oriented design principles:

- **Encapsulation:** Each class encapsulates its own data and behavior, ensuring controlled access and improved data integrity.
- **Abstraction:** Complex processes such as prediction and AI response generation are abstracted into dedicated service classes.
- **Modularity:** The system is divided into independent classes, making it easier to maintain and extend.
- **Separation of Concerns:** Each class has a clearly defined responsibility, reducing system complexity and improving maintainability.

Summary

The class diagram provides a structured representation of the PneumoAI system, clearly defining its core components and their relationships. It demonstrates how object-oriented principles are applied to create a modular, scalable, and maintainable system. By visualizing the static structure, this model serves as a blueprint for implementation and ensures that all system components are logically organized and interconnected.

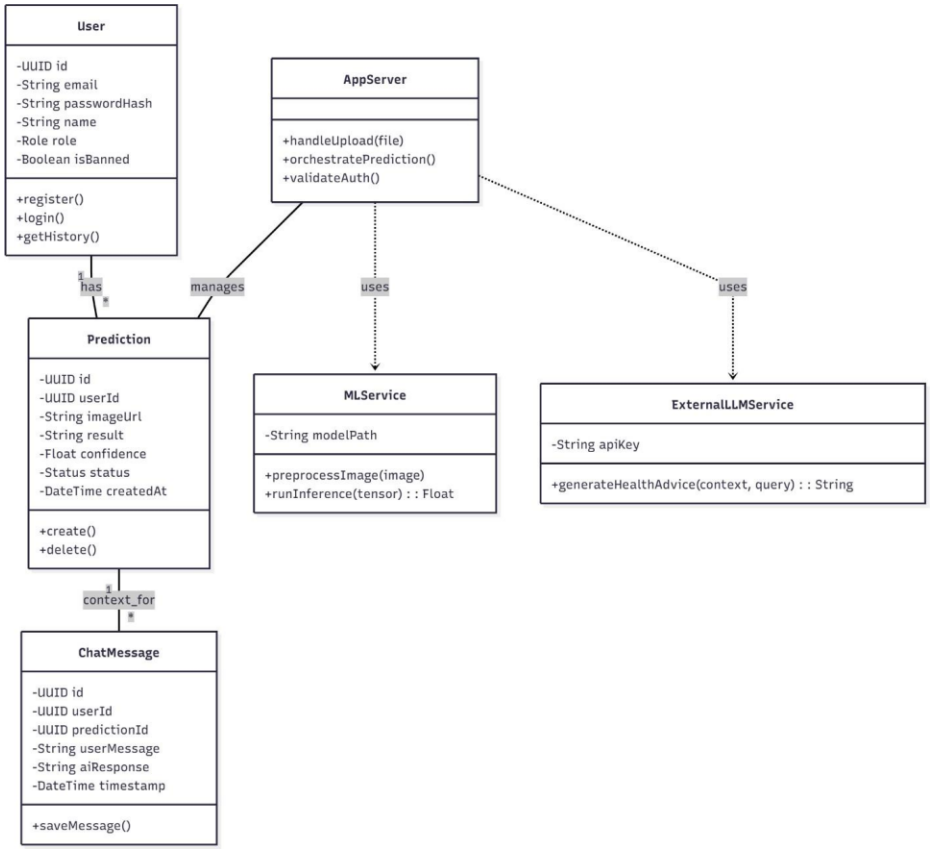

Figure 4.3: Class Diagram of PneumoAI

4.4.2. Sequence Diagram

The sequence diagram illustrates the dynamic interaction between different components of the PneumoAI system during execution. It provides a time-ordered view of how requests are processed, showing how data flows between the user, frontend, backend, storage services, machine learning service, and AI assistant. This diagram is essential in Software Engineering for understanding runtime behavior and communication between system components.

Main Flow Explanation

The sequence begins when the user uploads a chest X-ray image through the frontend interface. The frontend sends this data to the backend via an API request (POST /api/predictions). The backend first performs authentication and validates the uploaded file to ensure it meets the required format and size constraints.

Once validated, the backend stores the raw image in cloud storage (e.g., Cloudinary) and receives an image URL in return. This URL is then used to create a new prediction record in the database with a status marked as *pending*. This step ensures that the system maintains a record of the request before processing begins.

Next, the backend sends a request to the machine learning service, passing the image URL. The ML service preprocesses the image (e.g., resizing and grayscale conversion) and performs inference using a model based on Convolutional Neural Networks. After processing, the service returns the classification result (Normal or Pneumonia) along with a confidence score.

The backend receives this result and updates the corresponding record in the database, changing its status to *completed* and storing the prediction details. It then sends a JSON response back to the frontend containing the result, confidence score, and image URL. Finally, the frontend displays this information on the user dashboard in a clear and interpretable format.

AI Interaction Flow

In addition to the prediction process, the sequence diagram also represents the interaction with the AI assistant. After viewing the results, the user may submit a query through the chatbot interface. This query is sent from the frontend to the backend, which retrieves relevant context (such as the latest prediction result).

The backend then forwards the query and context to an external AI service powered by a Large Language Model. The AI service processes the input and generates a context-aware response. This response is returned to the backend and then sent to the frontend, where it is displayed to the user. The interaction may also be stored in the database for future reference.

Key Observations

- **Sequential Interaction:** The diagram clearly shows the step-by-step communication between system components, ensuring a logical and traceable workflow.
- **Service-Oriented Communication:** The backend acts as a central coordinator, interacting with both the ML service and the AI assistant service through APIs.

- **Asynchronous Processing:** The system allows non-blocking operations, particularly during image processing and AI response generation, improving overall performance.
- **Data Consistency:** The use of intermediate states (e.g., *pending*, *completed*) ensures that the system maintains consistent records throughout the process.

Summary

The sequence diagram provides a detailed view of how PneumoAI processes user requests and coordinates between multiple services. It highlights the interaction between the frontend, backend, cloud storage, database, machine learning service, and AI assistant. By visualizing the order of operations and data exchange, this diagram helps ensure that the system is logically structured, efficient, and capable of handling real-time interactions.

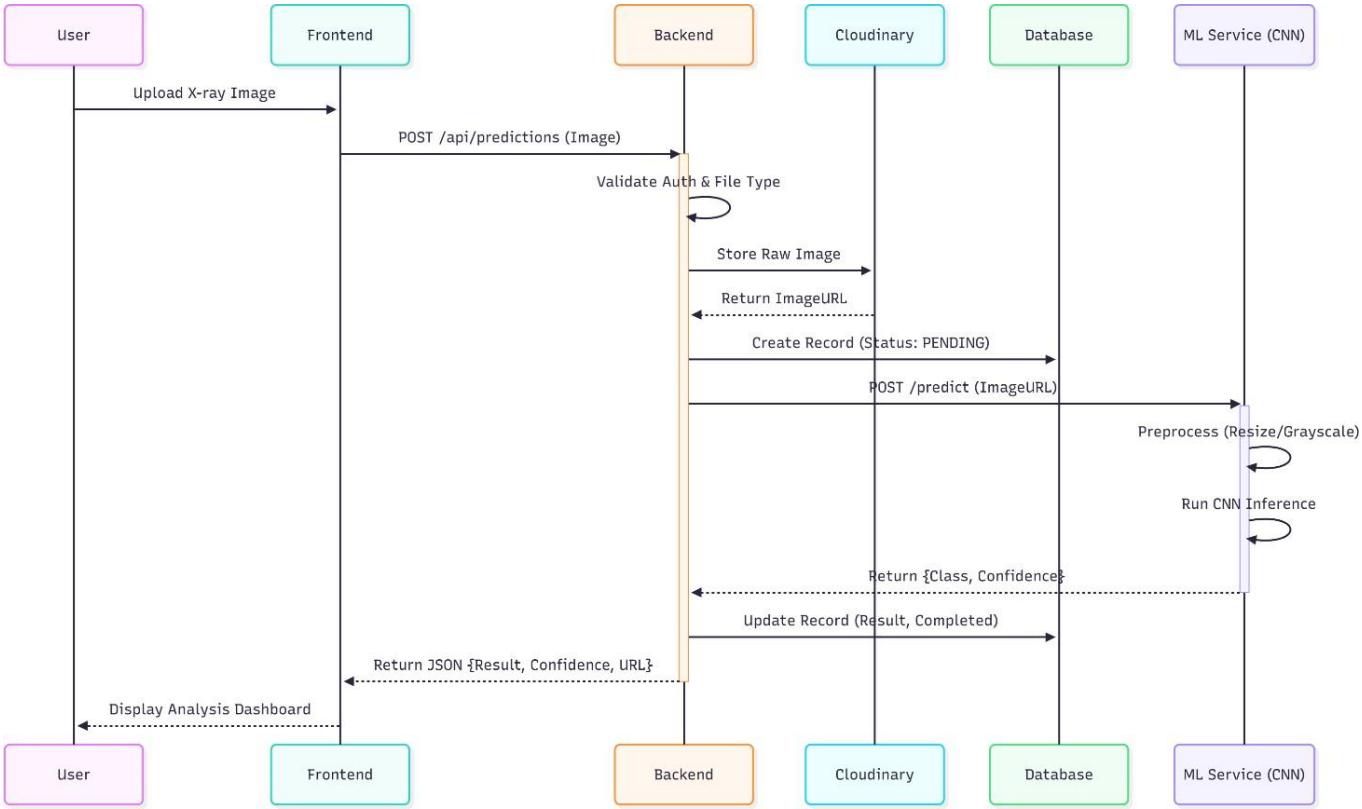

Figure 4.4: Sequence Diagram

4.4.3. State Diagram

The state diagram represents the lifecycle of a prediction process within the PneumoAI system. It illustrates how the system transitions between different states as an image moves from upload to final outcome. This model is essential in Software Engineering for understanding system behavior under different conditions and ensuring that each stage of processing is properly controlled and monitored.

Overview of System States

The diagram begins with the initial state, where the system is idle and waiting for user interaction. Once a user uploads an image, the system transitions into the **Uploading** state, indicating that the file has been received and is ready for validation.

The next stage is the **Validating** state, where the system performs checks on the uploaded file. This includes verifying the file format and size to ensure it meets system requirements. If the file fails validation, the system transitions to an **Error** state, preventing further processing and notifying the user.

If the file is valid, the system proceeds to the **Processing** state. In this phase, the image is stored in cloud storage, and the prediction request is sent to the machine learning service. The system then enters a waiting stage for inference, where the model based on Convolutional Neural Networks analyzes the image.

Prediction Outcome States

After analysis, the system transitions into one of several possible states depending on the result:

- **Completed:** This state represents a successful prediction with high confidence. The result (Normal or Pneumonia) is generated and made available to the user.
- **Flagged:** This state occurs when the prediction confidence is low or ambiguous. It indicates that the result may require further verification or professional review.
- **Service Error:** This state represents a failure in the machine learning service, such as a timeout or processing error.
- **Error:** This state is triggered when invalid input or system-level issues occur during earlier stages, such as file validation failure.

Chat Interaction State

An additional state, **Chat Active**, represents the user's interaction with the AI assistant after receiving a prediction. From the *Completed* state, users can initiate a chat session to ask questions and receive guidance. The system remains in this state until the user ends the interaction, after which it transitions to the final state.

State Transitions and Flow Control

The diagram demonstrates how the system handles both successful and unsuccessful scenarios through well-defined transitions:

- Validation determines whether the system proceeds to processing or returns an error.
- Processing leads to multiple outcomes based on prediction confidence or system performance.
- Error handling ensures that failures do not disrupt the entire system.
- Optional transitions allow users to engage with additional features such as the AI assistant.

Importance of State Management

The use of a state diagram provides several advantages:

- **Reliability:** Ensures that each stage of the process is controlled and predictable.
- **Error Handling:** Clearly defines how the system responds to invalid inputs and service failures.
- **Traceability:** Allows tracking of the system's current state at any point in time.
- **Scalability:** Supports extension of states for future features or enhancements.

Summary

The state diagram provides a detailed view of the lifecycle of a prediction process in PneumoAI, from image upload to final output and optional user interaction. By defining clear states and transitions, the system ensures robust handling of different scenarios, improving both reliability and user experience. This structured approach is essential for managing complex workflows in AI-driven applications.

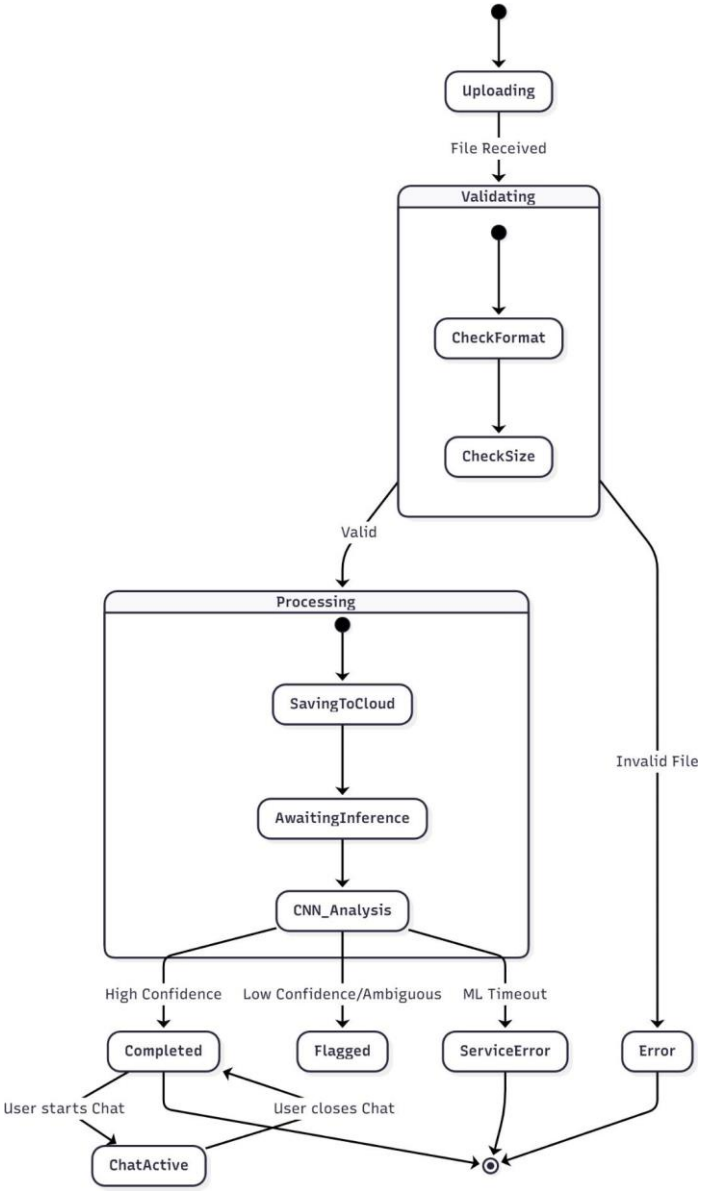

Figure 4.5: State Diagram

CHAPTER # 5

5. Implementation

This chapter presents the implementation details of the PneumoAI system, describing how the designed architecture is translated into a working application. It focuses on the development and integration of system components, demonstrating how each module contributes to achieving the overall functionality. Implementation is a crucial phase in Software Engineering, as it bridges the gap between theoretical design and practical execution.

The PneumoAI system is implemented using a full-stack approach that combines modern web technologies with artificial intelligence services. The system is structured into multiple interconnected components, including the frontend interface, backend server, machine learning service, and AI assistant integration. Each component is developed independently and then integrated through well-defined APIs to ensure smooth communication and efficient data flow.

The **frontend implementation** focuses on creating an intuitive and responsive user interface that allows users to interact with the system. It includes features such as user authentication, image upload, result visualization, and chatbot interaction. The design ensures that users can perform all required actions with minimal effort while maintaining clarity and accessibility.

The **backend implementation** serves as the core of the system, handling business logic, request processing, and communication between components. It manages user authentication, validates input data, processes API requests, and coordinates interactions with both the machine learning service and the AI assistant. The backend also ensures secure data handling and efficient system performance.

A key part of the implementation is the integration of the machine learning model, which is responsible for pneumonia detection. The model, based on Convolutional Neural Networks, is deployed as a separate service to allow independent processing and scalability. This service receives image data, performs preprocessing and inference, and returns prediction results to the backend.

In addition to image-based prediction, the system integrates a conversational AI component powered by a Large Language Models. This integration enables users to interact with the system using natural language, providing context-aware guidance related to Pneumonia. The backend manages this interaction by constructing appropriate prompts and handling responses from the external API.

The implementation also includes algorithmic representations of major modules, outlining the step-by-step logic followed during operations such as image processing, prediction generation, and chatbot interaction. These algorithms provide a clear understanding of how the system processes input data and produces output results.

Furthermore, this chapter discusses the use of external APIs and services that support system functionality, including cloud storage for image handling and AI APIs for conversational interaction. These integrations enhance the system’s capabilities while reducing the need for building complex components from scratch.

An overview of the user interface is also included, highlighting key screens and interactions that define the user experience. The interface is designed to be simple, responsive, and accessible, ensuring that users can easily navigate the system and understand the results provided.

Overall, this chapter demonstrates how the PneumoAI system is implemented as a cohesive solution, combining multiple technologies and components into a unified application. It provides detailed insights into the development process, showcasing how the system's design is realized in practice and how it effectively delivers its intended functionality.

5.1. Algorithm

The PneumoAI system is composed of multiple interconnected modules that work together to deliver end-to-end functionality, including backend processing, machine learning inference, AI assistant interaction, and image storage handling. The following algorithms describe the logical flow of these core components. These algorithmic representations provide a clear understanding of how input data is processed and transformed into meaningful outputs within the system.

Algorithm 1: Prediction Processing (Backend API)

Input: X-ray image file, User ID

Output: Prediction result and confidence score

Steps:

1. Receive the image file from the user through the frontend interface.
2. Validate the file type (e.g., JPG, PNG, DICOM) and ensure it meets size constraints.
3. Store the validated image in cloud storage and generate a public URL.
4. Create a new prediction record in the database with status set to *"Pending"*.
5. Send the image URL to the machine learning microservice for analysis.
6. Receive the prediction result and confidence score from the ML service.
7. Update the prediction record in the database with the result and status set to *"Completed"*.
8. Return the prediction result and confidence score to the frontend for display.

This algorithm acts as the central orchestration layer, coordinating communication between storage, AI services, and the database.

Algorithm 2: CNN Model Prediction (ML Service)

Input: X-ray image

Output: Classification (Normal/Pneumonia) and confidence

Steps:

1. Load the input image from the provided source.
2. Convert the image to grayscale to simplify feature extraction.
3. Resize the image to a fixed dimension (e.g., 224 × 224 pixels) to match model input requirements.
4. Normalize pixel values to ensure consistent scaling.
5. Pass the processed image through a trained model based on Convolutional Neural Networks.
6. Apply a softmax function to generate probability scores for each class.
7. Compare the probabilities against a predefined threshold.
8. Return the predicted class (Normal or Pneumonia) along with the confidence score.

This algorithm represents the core AI functionality responsible for medical image analysis and classification.

Algorithm 3: AI Health Assistant Response

Input: User query, Prediction context

Output: AI-generated response

Steps:

1. Retrieve the user's latest prediction result and confidence score.
2. Construct a controlled system prompt with restrictions, including:
    - Only pneumonia-related responses
    - No medical prescriptions or unsafe advice
3. Combine the user query with the contextual information.
4. Send the request to an external API powered by a Large Language Models.
5. Receive the generated response from the AI service.
6. Filter and sanitize the response to ensure relevance and safety.
7. Return the processed response to the user via the frontend interface.

This algorithm ensures that the chatbot provides meaningful, safe, and context-aware guidance.

Algorithm 4: Image Storage Handling

Input: Image file

Output: Public image URL

Steps:

1. Generate a unique file name to avoid conflicts.
2. Validate the image format and ensure it meets system requirements.
3. Upload the image to a cloud storage service.
4. Store the image securely with appropriate access controls.
5. Generate and return a public URL for accessing the image.

This algorithm supports efficient handling of unstructured data and enables seamless integration with the prediction pipeline.

Overall System Perspective

These algorithms collectively define the operational logic of PneumoAI. The backend algorithm coordinates the workflow, the CNN-based model performs intelligent analysis, the AI assistant enhances user interaction, and the storage mechanism ensures efficient data handling. Together, they form a cohesive system that transforms user input into accurate predictions and informative responses.

5.2. External APIs

The PneumoAI system integrates external APIs for AI functionality and storage services.

Table 5.1: Details of APIs Used in the Project

Name of API	Description	Purpose of Usage	Functions/Modules
OpenAI / HuggingFace API	Provides Large Language Model for chatbot responses	Generate pneumonia-related guidance	AI Assistant Module
Cloudinary API	Cloud-based image storage service	Store and retrieve X-ray images	Image Upload Module
REST API (Custom Backend)	Handles communication between frontend and backend	Manage requests and responses	All system modules

5.3. User Interface

The user interface of PneumoAI is designed to be intuitive, responsive, and user-friendly, ensuring smooth interaction for both technical and non-technical users. A strong emphasis is placed on usability and clarity, following principles from Human-Computer Interaction. The interface is developed using modern frontend

technologies to provide a clean layout, fast performance, and consistent user experience across devices.

The UI is structured into multiple components, each responsible for a specific functionality within the system. These components collectively ensure that users can easily navigate the application, perform tasks efficiently, and understand system outputs without confusion.

1. Login and Registration Interface

The authentication interface serves as the entry point to the system. It allows users to either create a new account or log in using existing credentials.

- Users can securely register using email and password.
- Login functionality validates user credentials and grants access to the system.
- Error messages are displayed clearly for invalid inputs, such as incorrect passwords or missing fields.
- The interface follows a minimal and clean design, reducing cognitive load and improving usability.

This component ensures secure and straightforward access to the application.

2. Dashboard Interface

The dashboard acts as the central hub of the system, providing access to all major functionalities.

- It displays key options such as image upload, prediction history, and chatbot access.
- Navigation elements are clearly organized, allowing users to switch between features easily.
- Previous prediction records are displayed in a structured format for quick reference.
- The layout is designed to prioritize important actions, improving user efficiency.

The dashboard enhances user experience by centralizing all functionalities in a single, accessible location.

3. Image Upload Interface

The image upload interface is a critical component that enables users to submit chest X-ray images for analysis.

- Users can upload images באמצעות drag-and-drop functionality or file selection.
- A preview of the selected image is displayed before submission, allowing users to verify correctness.
- The upload button triggers the prediction process, providing immediate feedback to the user.
- Validation messages guide users in case of unsupported formats or file size issues.

This interface is designed to be simple and efficient, minimizing errors during input.

4. Result Display Interface

The result interface presents the output generated by the system in a clear and interpretable format.

- Displays the prediction result (Normal or Pneumonia).
- Shows the confidence score using visual indicators such as progress bars or percentage values.
- Provides an option to download the result as a PDF report for further use.
- The layout ensures that critical information is highlighted and easy to understand.

This component plays a key role in ensuring transparency and usability of AI-generated results.

5. AI Chat Interface

The AI chat interface enables interaction with the system's conversational assistant.

- Users can ask questions related to pneumonia and receive context-aware responses.
- Responses are displayed in a chat format, making interaction natural and intuitive.
- Conversation history is maintained, allowing users to review previous interactions.
- The interface supports continuous interaction, enhancing user engagement.

This component transforms the system from a diagnostic tool into an interactive support platform.

6. Admin Interface

The admin interface is designed for system management and monitoring.

- Allows administrators to monitor system usage and user activity.
- Provides tools to manage user accounts, including viewing and restricting access.
- Displays system statistics and logs for performance tracking.
- Ensures secure access to administrative features through role-based control.

This interface supports system maintenance and ensures smooth operation.

Design Considerations

The user interface of PneumoAI is developed with the following key considerations:

- **Responsiveness:** The system adapts to different screen sizes and devices.

- **Consistency:** Uniform design elements ensure a cohesive user experience.
- **Accessibility:** Clear layouts and readable text improve usability for diverse users.
- **Feedback Mechanisms:** Users receive immediate responses to actions (e.g., errors, success messages).

Summary

The user interface of PneumoAI is carefully designed to provide a seamless and efficient user experience. By combining simplicity, responsiveness, and functionality, the system ensures that users can interact with complex AI features in an intuitive manner. This user-centered design approach enhances both usability and overall system effectiveness.

5.4. Screen Images

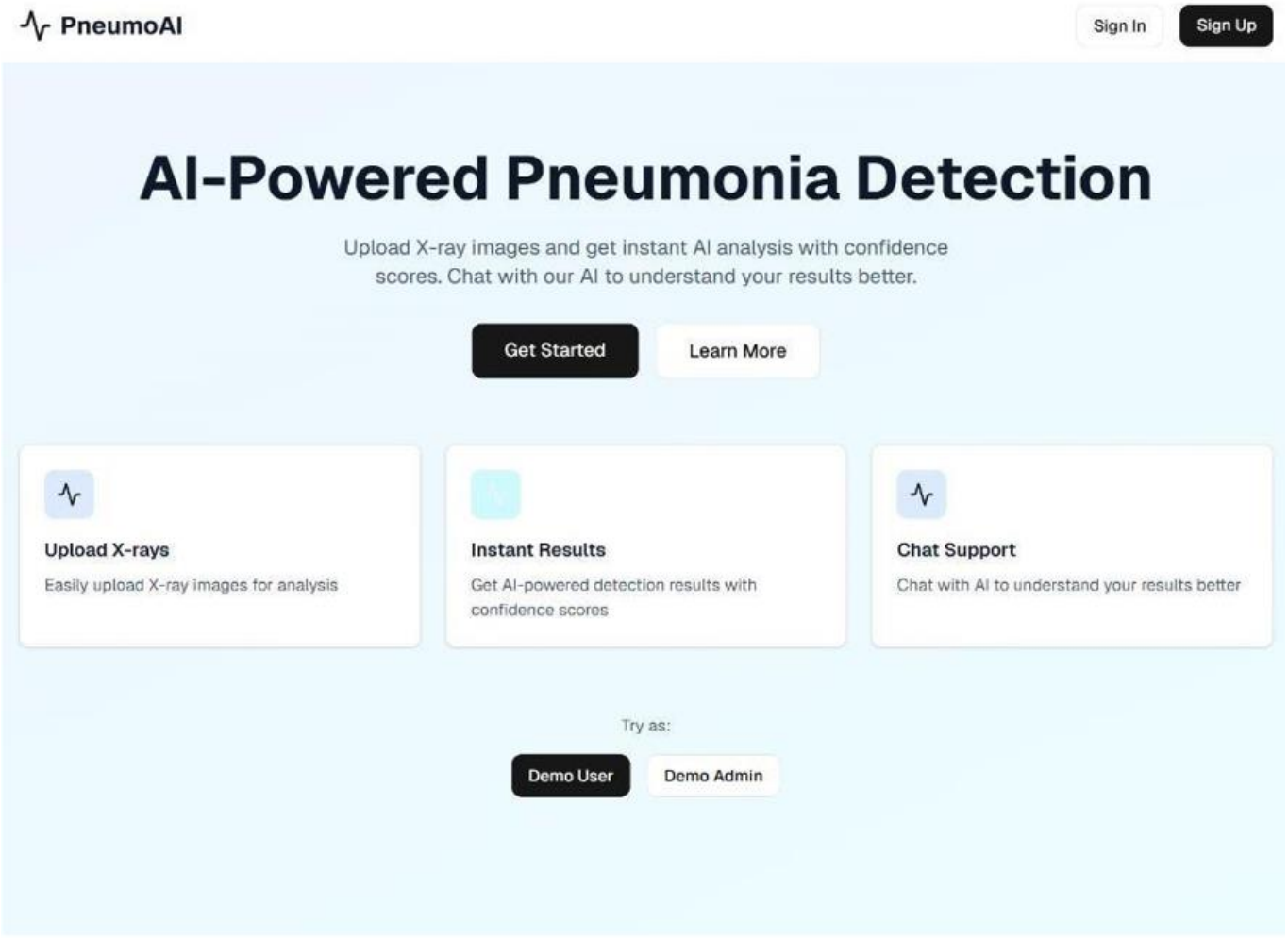

Figure 5.1: Home Screen

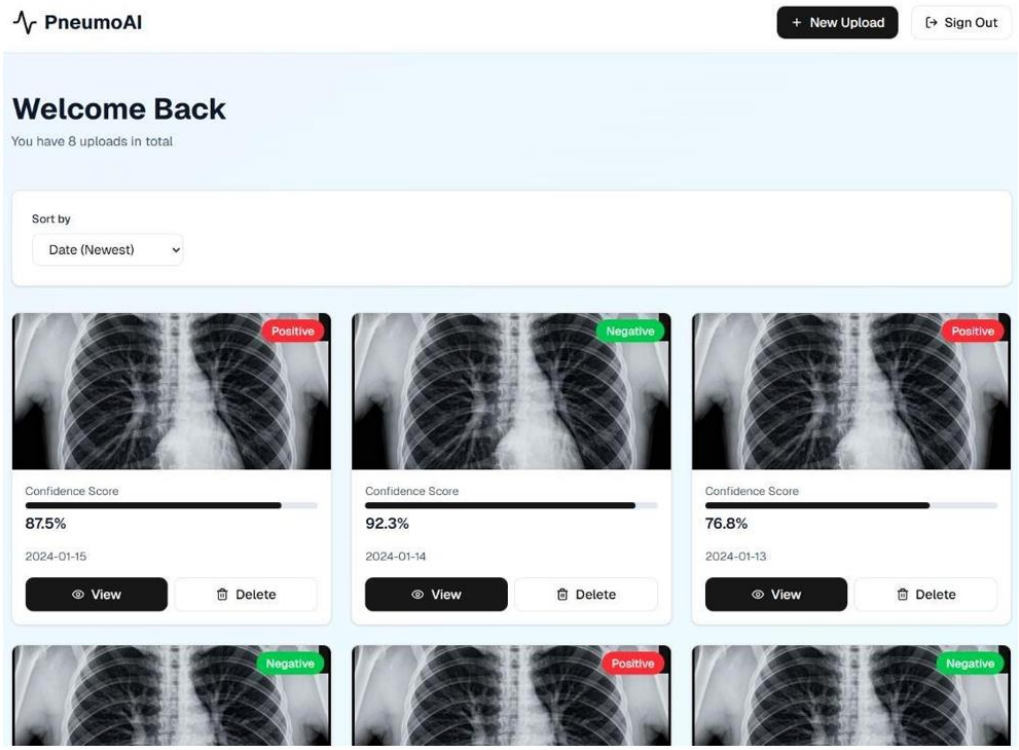

Figure 5.2: User Dashboard Screen

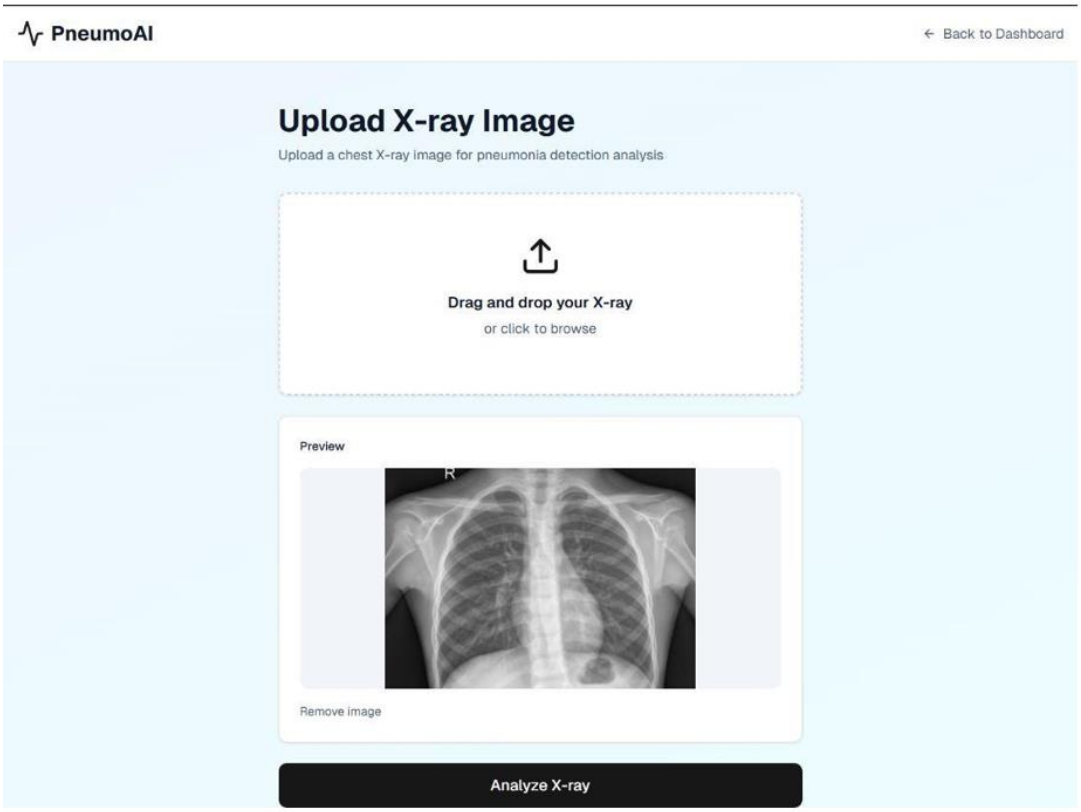

Figure 5.3: Image Upload Screen

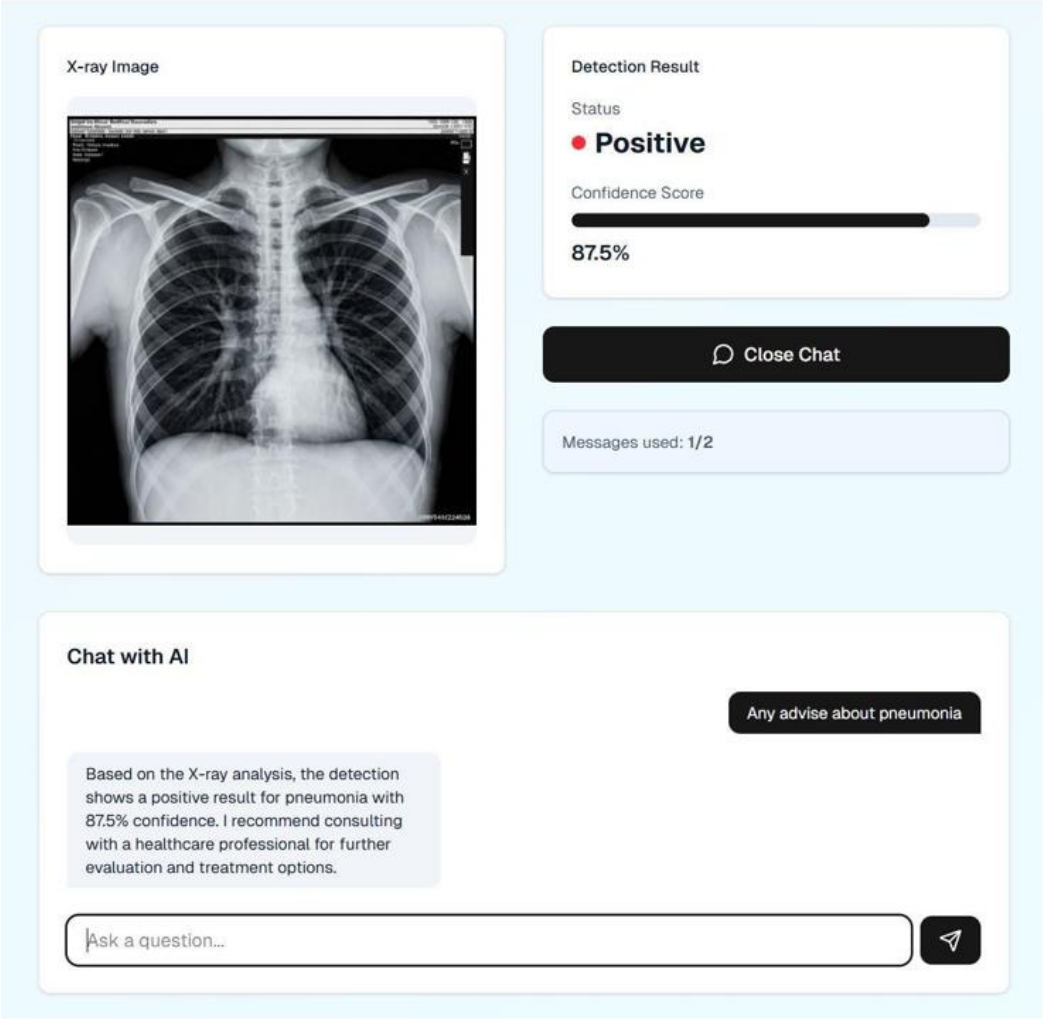

Figure 5.4: Result & Chat Screen

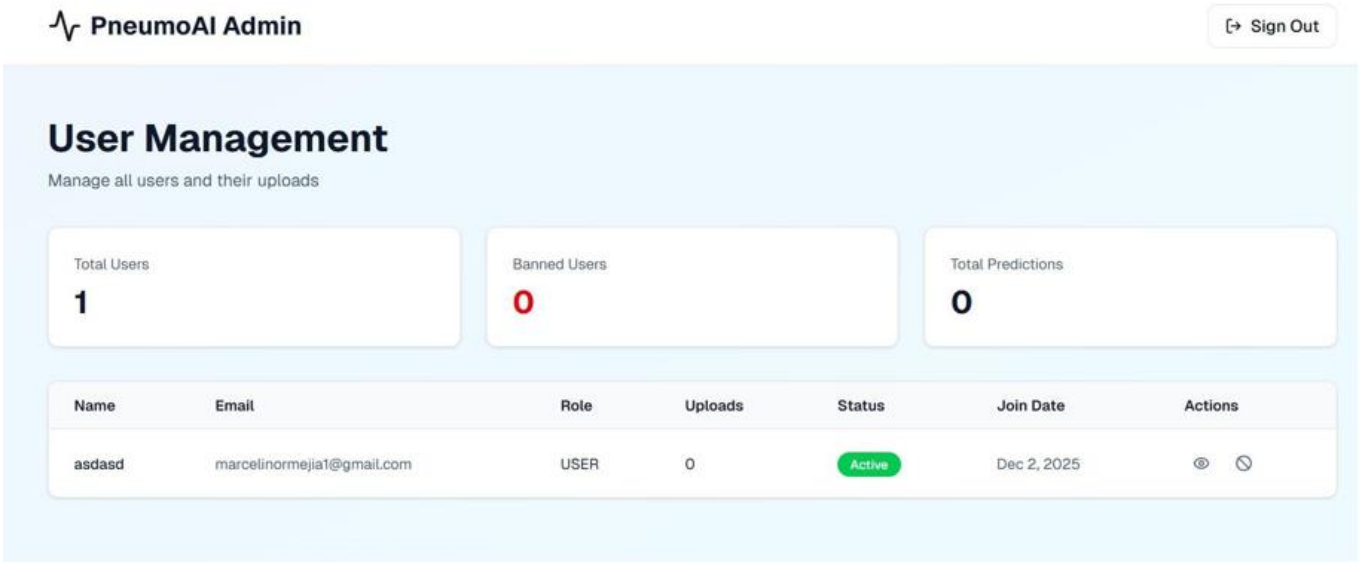

Figure 5.5: Admin Dashboard

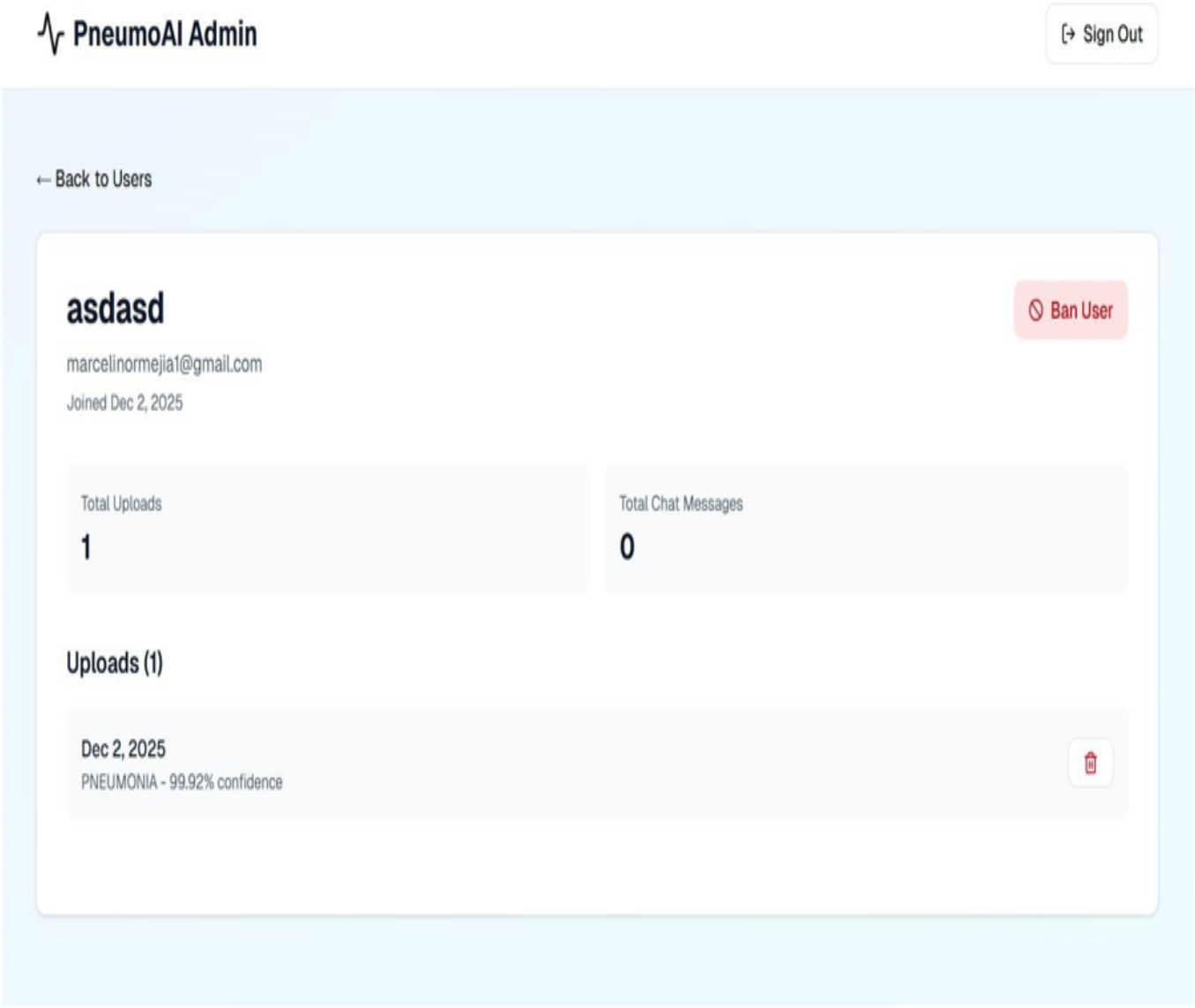

Figure 5.6: Admin User Management Screen

CHAPTER # 6

6. Testing and Evaluation

This chapter presents the testing and evaluation of the PneumoAI system to ensure that it functions correctly, reliably, and in accordance with the specified requirements. Testing is a critical phase in Software Engineering, as it verifies that the implemented system behaves as expected under different conditions and identifies potential issues before deployment.

The evaluation of PneumoAI is conducted using both **manual and automated testing approaches**, ensuring comprehensive validation of system functionality, performance, and robustness. These testing strategies are applied across multiple levels to cover all aspects of the system, including frontend interaction, backend processing, AI model performance, and integration between components.

Testing Approach Overview

The testing process is structured to validate the system at different levels:

- **Unit Testing:** Individual components of the system, such as authentication modules, API endpoints, and data processing functions, are tested independently. This ensures that each unit performs its intended function correctly in isolation.
- **Functional Testing:** The system is tested against its functional requirements to verify that all features such as image upload, prediction generation, chatbot interaction, and report download operate as expected. This type of testing focuses on user-level interactions and system behavior.
- **Integration Testing:** This level of testing ensures that different system components such as the frontend, backend, machine learning service, and AI assistant work together seamlessly. It validates the communication between services and ensures that data flows correctly across the system.

System Validation Objectives

The testing and evaluation process aims to achieve the following objectives:

- Verify that all system functionalities are implemented correctly.
- Ensure that the system performs efficiently under normal operating conditions.
- Validate the accuracy and reliability of the AI-based prediction model.
- Confirm that the system handles errors and exceptional cases gracefully.
- Ensure secure handling of user data and system interactions.

Testing Scope

The scope of testing in PneumoAI includes:

- **Frontend Testing:** Validating user interface components, navigation flow, responsiveness, and usability.

- **Backend Testing:** Testing API endpoints, authentication mechanisms, and data processing logic.
- **AI Model Evaluation:** Assessing the performance of the model based on Convolutional Neural Networks, including prediction accuracy and confidence reliability.
- **Chatbot Testing:** Evaluating responses generated by the AI assistant powered by a Large Language Models, ensuring relevance and contextual accuracy.
- **Database Testing:** Verifying correct storage, retrieval, and integrity of data within the system.

Evaluation Perspective

In addition to testing functionality, the system is evaluated based on key performance indicators such as response time, system stability, and user experience. The evaluation also considers how effectively the system meets its intended purpose assisting in early detection of Pneumonia and providing meaningful user guidance.

Summary

Overall, this chapter ensures that PneumoAI is not only functionally correct but also reliable, efficient, and user-friendly. By applying multiple testing techniques and evaluating system performance, the project demonstrates its readiness for real-world usage and highlights its effectiveness as an AI-driven healthcare solution.

6.1. Manual Testing

Manual testing is conducted to verify that individual components and the overall PneumoAI system behave as expected under different scenarios. This type of testing involves direct interaction with the system by testers, simulating real user behavior to identify functional issues, usability problems, and unexpected system responses. Unlike automated testing, manual testing provides valuable insights into user experience and helps detect issues that may not be captured through scripted tests.

In the context of PneumoAI, manual testing focuses on validating core functionalities such as user authentication, image upload, prediction processing, chatbot interaction, and report generation. The objective is to ensure that all features operate correctly and align with the functional requirements defined earlier.

Testing Approach

Manual testing is performed using predefined test cases that cover both typical and edge-case scenarios. Each test case includes input conditions, expected outcomes, and actual results. The system is tested across different user flows to ensure consistency and reliability.

The testing process includes:

- Executing user actions such as registration, login, and navigation

- Uploading valid and invalid image files to test validation mechanisms
- Verifying prediction results generated by the AI model
- Interacting with the chatbot to assess response accuracy and relevance
- Testing report generation and download functionality
- Evaluating system behavior under error conditions

Sample Test Cases

Table 6.1: Sample Test Cases for Manual Testing

Test Case ID	Test Scenario	Input	Expected Output	Actual Result	Status
TC-01	User Registration	Valid email & password	Account created successfully	As expected	Pass
TC-02	User Login	Correct credentials	User logged in and redirected to dashboard	As expected	Pass
TC-03	Invalid Login	Incorrect password	Error message displayed	As expected	Pass
TC-04	Image Upload (Valid)	JPG X-ray image	Image uploaded successfully	As expected	Pass
TC-05	Image Upload (Invalid Format)	Unsupported file type	Upload rejected with error message	As expected	Pass
TC-06	Prediction Processing	Valid X-ray image	Prediction result with confidence displayed	As expected	Pass
TC-07	Chatbot Interaction	Pneumonia-related query	Relevant AI response displayed	As expected	Pass
TC-08	Report Generation	Valid prediction result	PDF report downloaded	As expected	Pass
TC-09	System Error Handling	ML service unavailable	Fallback message displayed	As expected	Pass

Observations

- The system successfully handled all standard user interactions without major issues.
- Validation mechanisms effectively prevented incorrect inputs, such as invalid file formats.
- The prediction workflow functioned correctly, returning results within expected time limits.
- The chatbot provided relevant and context-aware responses in most cases.
- Error handling mechanisms ensured that failures did not disrupt the entire system.

Limitations of Manual Testing

While manual testing is effective for identifying usability issues and verifying functionality, it has certain limitations:

- It can be time-consuming and less efficient for repetitive test cases.
- It may not cover all possible edge cases or performance-related scenarios.
- Results may vary depending on the tester's approach and experience.

Summary

Manual testing plays a vital role in validating the PneumoAI system from a user perspective. It ensures that the system behaves correctly under real-world conditions and provides a smooth and reliable user experience. The successful execution of test cases indicates that the system meets its functional requirements and is ready for further evaluation through automated testing and performance analysis.

6.2. System Testing

System testing is conducted to validate the PneumoAI system as a whole, ensuring that all integrated components function together correctly and meet both functional and non-functional requirements. Unlike unit or module-level testing, system testing focuses on **end-to-end validation**, verifying that the complete workflow from user interaction to AI-driven output operates seamlessly.

In PneumoAI, system testing evaluates how different modules such as authentication, image processing, prediction generation, database operations, and AI chatbot interaction work together as a unified system. This level of testing is essential in Software Engineering to ensure that integration between components does not introduce errors or inconsistencies.

Objectives of System Testing

The primary objectives of system testing include:

- Verifying that all system functionalities operate as expected when integrated

- Ensuring smooth communication between frontend, backend, and AI services
- Validating system performance under normal operating conditions
- Confirming that non-functional requirements such as security, usability, and reliability are satisfied
- Identifying any issues arising from module interaction

System Testing Scenarios

The following scenarios were tested to validate the complete system workflow:

Table 6.2: System Testing Scenario

Test Scenario	Description	Expected Outcome	Result
End-to-End Prediction Flow	User logs in → uploads image → receives prediction	Correct prediction displayed with confidence	Pass
Authentication Integration	User login with backend validation and session handling	Secure login and dashboard access	Pass
Image Processing Pipeline	Image upload → validation → storage → ML processing	Image processed successfully without errors	Pass
AI Chatbot Integration	User query → backend → LLM → response	Relevant and context-aware response returned	Pass
Database Integration	Prediction and chat data stored and retrieved	Data stored accurately and displayed correctly	Pass
Error Handling	Invalid input or service failure	Appropriate error message displayed	Pass
Report Generation	Generate and download prediction report	PDF report generated correctly	Pass

Performance and Behavior Validation

During system testing, the performance of key components was evaluated:

- Prediction results were generated within acceptable time limits (approximately 3–5 seconds).
- Chatbot responses were delivered quickly and remained contextually relevant.

- The system maintained stability during continuous usage and multiple requests.
- No critical failures were observed during normal operation.

Integration Validation

System testing confirmed that all major components interact effectively:

- The frontend successfully communicates with the backend through API requests.
- The backend correctly coordinates with the machine learning service based on Convolutional Neural Networks for prediction processing.
- The backend integrates with the AI assistant powered by a Large Language Models to provide chatbot responses.
- Data flows consistently between the database, cloud storage, and application layers.

Observations

- The system demonstrated stable and reliable performance across all tested scenarios.
- Integration between modules was smooth, with no major communication failures.
- The user workflow from login to prediction and chatbot interaction was seamless.
- Error handling mechanisms effectively prevented system crashes and ensured user feedback.

Summary

System testing confirms that PneumoAI operates as a fully integrated and functional system. All modules work together cohesively, meeting both functional and non-functional requirements. The successful execution of end-to-end workflows indicates that the system is reliable, efficient, and ready for deployment in real-world scenarios.

6.3. Unit Testing

Unit Testing 1: User Login

Testing Objective: To verify that the login functionality works correctly.

Table 6.3: Unit Testing 1

Test Case / Script	Input	Expected Result	Result
Login with valid	Email: user@test.com	User successfully logs in and	Pass

Test Case / Script	Input	Expected Result	Result
credentials	Password: 123456	dashboard is displayed	
Login with invalid credentials	Email: user@test.com Password: wrong	Error message displayed	Pass

Unit Testing 2: Image Upload

Testing Objective: To verify image upload functionality.

Table 6.4: Unit Testing 2

Test Case / Script	Input	Expected Result	Result
Upload valid image	JPG file (≤10MB)	Image uploaded successfully	Pass
Upload invalid file type	PDF file	Error message displayed	Pass

Unit Testing 3: Prediction Module

Testing Objective: To verify CNN prediction processing.

Table 6.5: Unit Testing 3

Test Case / Script	Input	Expected Result	Result
Valid X-ray image	Chest X-ray	Prediction result displayed	Pass
Corrupted image	Invalid file	Error handled gracefully	Pass

6.4. Functional Testing

Functional testing ensures that each module performs according to the system requirements.

Functional Testing 1: User Authentication

Objective: To ensure correct navigation after login.

Table 6.6: Functional Testing 1

Test Case	Input	Expected Result	Result
Login as user	Valid credentials	User dashboard displayed	Pass

Functional Testing 2: Prediction Workflow

Objective: To verify end-to-end prediction process.

Table 6.7: Functional Testing 2

Test Case	Input	Expected Result	Result
Upload and analyze image	Valid X-ray	Result + confidence displayed	Pass

Functional Testing 3: AI Chatbot

Objective: To verify chatbot responses.

Table 6.8: Functional Testing 3

Test Case	Input	Expected Result	Result
Ask pneumonia-related question	"What are symptoms?"	Relevant response generated	Pass
Ask unrelated question	"What is football?"	Restricted response or warning	Pass

6.5. Integration Testing

Integration testing verifies the interaction between different modules.

Table 6.9: Integration Testing

Test Case	Description	Expected Result	Result
Login + Upload + Prediction	Full workflow	Successful execution	Pass

Test Case	Description	Expected Result	Result
Prediction + Chatbot	Context-based response	AI response relevant to result	Pass
Upload + Storage + Retrieval	Image stored and retrieved	Image URL accessible	Pass

6.6. Automated Testing

Automated testing tools are used to validate system performance and API responses.

Tools Used

Table 6.10: Tools Used in Automated Testing

Tool Name	Description	Applied On	Results
Postman	API testing tool	Backend APIs (Login, Prediction, Chat)	All APIs responded correctly
Jest	JavaScript testing framework	Backend functions	Passed
Browser DevTools	Performance testing	UI performance	Within acceptable limits

6.7. Evaluation Summary

The evaluation of the PneumoAI system demonstrates that it performs reliably and effectively across all major modules. Comprehensive testing, including manual, functional, and system-level validation, confirms that the system operates as intended and meets the requirements defined in earlier chapters. Each core component ranging from user authentication to AI-driven prediction and chatbot interaction has been successfully tested and validated.

From a functional perspective, all key features of the system are working correctly. Users are able to register, log in securely, upload chest X-ray images, receive prediction results, interact with the AI assistant, and download reports without issues. The integration between the frontend, backend, machine learning service, and AI assistant is seamless, ensuring a smooth end-to-end workflow. This confirms that the system design and implementation are consistent with the specified functional requirements.

In terms of performance, the system meets the defined benchmarks. Prediction results generated using the model based on Convolutional Neural Networks are delivered within acceptable time limits, and chatbot

responses powered by Large Language Models are generated efficiently. The system remains stable under normal load conditions, supporting multiple users without significant performance degradation.

Reliability and robustness are also evident from the testing results. The system handles errors gracefully, providing appropriate feedback in cases such as invalid input, service unavailability, or network issues. These mechanisms ensure that failures in one component do not disrupt the overall system functionality, thereby improving user experience and system dependability.

Security requirements have been successfully implemented and validated. User data is protected through secure authentication mechanisms, encrypted communication protocols, and controlled access to sensitive features. This ensures that the system maintains confidentiality and integrity of user information, which is particularly important in healthcare-related applications.

Overall, the evaluation confirms that PneumoAI meets both functional and non-functional requirements. The system demonstrates strong performance, stability, and usability, making it suitable for real-world deployment. While further enhancements and optimizations may be considered in future iterations, the current implementation provides a reliable and effective solution for AI-assisted pneumonia detection and user guidance.

6.8. Results

This chapter presents the experimental results obtained from the deep learning models developed for pneumonia detection using chest X-ray images. Three different models were evaluated: a Convolutional Neural Network (CNN), a Bidirectional Long Short-Term Memory (BiLSTM) model, and a Hybrid CNN+BiLSTM model. The performance of each model was assessed using standard evaluation metrics including Accuracy, Precision, Recall, F1-Score, and Area Under the ROC Curve (AUC). In addition, confusion matrices were analyzed to evaluate the classification performance of each model and identify correctly and incorrectly classified samples.

The objective of these experiments was to determine the most effective model for detecting pneumonia while maintaining high reliability and minimizing classification errors.

6.8.1. CNN Model Results

The CNN model demonstrated excellent performance on the test dataset. It achieved an accuracy of **96.93%**, indicating that it correctly classified the majority of chest X-ray images. The model also obtained a precision of **96.99%**, recall of **96.93%**, F1-score of **96.89%**, and an AUC value of **99.58%**, showing outstanding discriminative capability.

The confusion matrix further illustrates the model's effectiveness. Out of all test samples, **143 normal** images and **426 pneumonia** images were correctly classified. Only **16 normal** images were incorrectly classified as pneumonia, while **2 pneumonia** images were classified as normal. These results indicate that the CNN model successfully identifies pneumonia cases with a very low false-negative rate, which is particularly important in medical diagnosis.

```
# CNN

Test Accuracy: 0.9693
Test Precision: 0.9699
Test Recall: 0.9693
Test F1-Score: 0.9689
Test AUC: 0.9958
CPU times: user 1.01 s, sys: 2.03 ms, total: 1.02 s
Wall time: 1.02 s
```

Figure 6.1: CNN Model Result

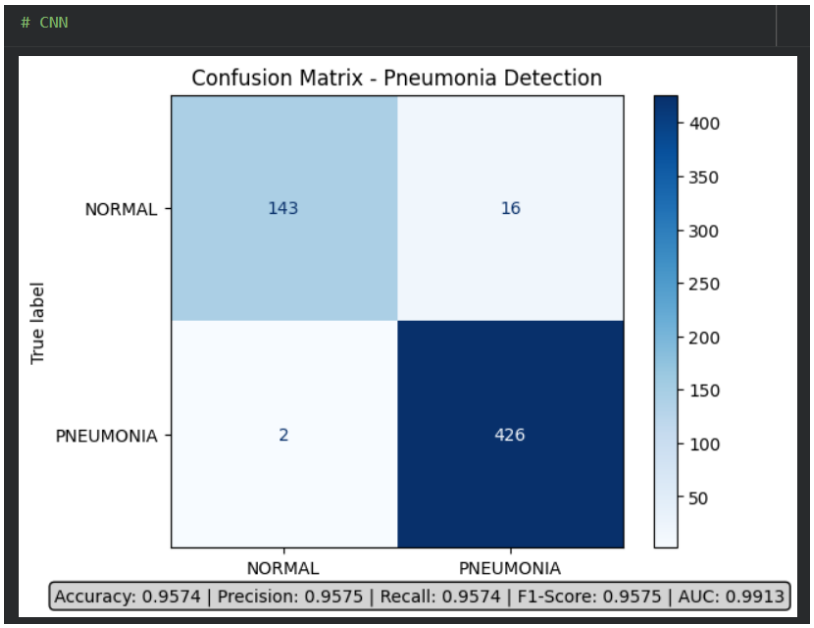

Figure 6.2: CNN Model Confusion Matrix

6.8.2. BiLSTM Model Results

The BiLSTM model was also evaluated on the same test dataset. It achieved an accuracy of **93.70%**, with a precision of **94.07%**, recall of **93.70%**, F1-score of **93.79%**, and an AUC value of **98.21%**.

The confusion matrix shows that the model correctly classified **149 normal** images and **401 pneumonia** images. However, it misclassified **10 normal** images as pneumonia and **27 pneumonia** images as normal. Compared with the CNN model, the BiLSTM produced a larger number of false negatives, indicating that it was comparatively less effective for pneumonia detection.

```
📊 BiLSTM — Test Results
  Accuracy:  0.9370
  Precision: 0.9407
  Recall:    0.9370
  F1-Score:  0.9379
  AUC:       0.9821
CPU times: user 241 ms, sys: 1.01 ms, total: 242 ms
Wall time: 242 ms
```

Figure 6.3: BiLSTM Model Results

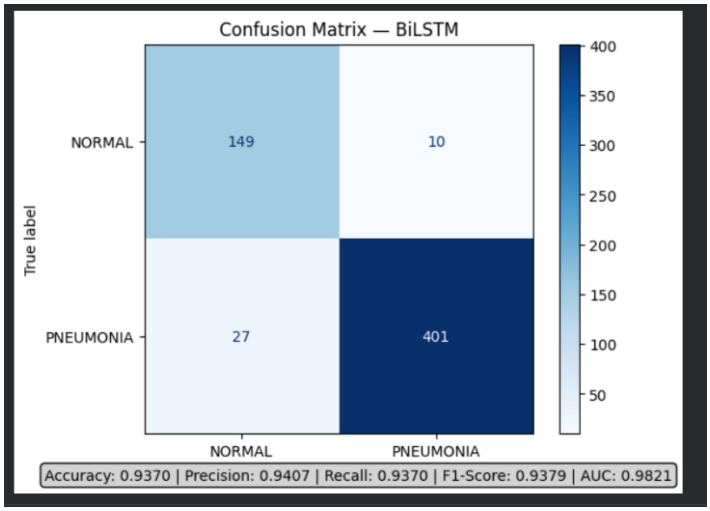

Figure 6.4: BiLSTM Confusion Matrix

6.8.3. CNN+BiLSTM Hybrid Model Results

The hybrid CNN+BiLSTM model combined the feature extraction capability of CNN with the sequence learning capability of BiLSTM. This model achieved the highest overall performance among all evaluated models.

The hybrid model obtained an accuracy of **97.10%**, precision of **97.13%**, recall of **97.10%**, F1-score of **97.11%**, and an AUC of **99.22%**.

The confusion matrix shows that **152 normal** images and **418 pneumonia** images were correctly classified. Only **7 normal** images were misclassified as pneumonia, while **10 pneumonia** images were predicted as normal. These results demonstrate that the hybrid architecture provides a balanced and highly accurate classification performance.

```
📊 CNN+BiLSTM Hybrid — Test Results
  Accuracy:  0.9710
  Precision: 0.9713
  Recall:    0.9710
  F1-Score:  0.9711
  AUC:       0.9922
CPU times: user 969 ms, sys: 6.01 ms, total: 975 ms
Wall time: 976 ms
```

Figure 6.5: CNN + BiLSTM Hybrid Results

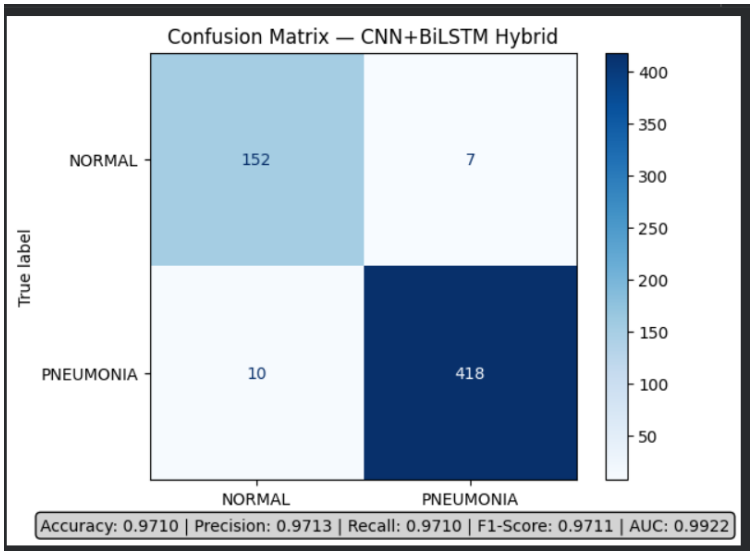

Figure 6.6: CNN + BiLSTM Hybrid Confusion Matrix

6.8.4. Comparative Analysis of Models

To identify the most suitable model for pneumonia detection, the performance of all three models was compared using the evaluation metrics Accuracy, Precision, Recall, F1-Score, and AUC.

Model	Accuracy	Precision	Recall	F1-Score	AUC
CNN	96.93%	96.99%	96.93%	96.89%	99.58%
BiLSTM	93.70%	94.07%	93.70%	93.79%	98.21%
CNN+BiLSTM	**97.10%**	**97.13%**	**97.10%**	**97.11%**	**99.22%**

The comparison demonstrates that the **CNN+BiLSTM Hybrid model achieved the highest values for Accuracy, Precision, Recall, and F1-Score**, indicating superior overall classification performance.

Although the CNN model produced a slightly higher AUC value (**99.58%**) compared to the hybrid model (**99.22%**), the hybrid model consistently performed better across the remaining evaluation metrics. Therefore, it provides a better balance between sensitivity and overall predictive performance.

The BiLSTM model produced comparatively lower scores across all evaluation metrics, suggesting that it is less suitable than CNN-based approaches for this pneumonia detection task.

```
# Model Comparison


📊 CNN — Test Results
  Accuracy:  0.9693
  Precision: 0.9699
  Recall:    0.9693
  F1-Score:  0.9689
  AUC:       0.9958

======================================================================
📊 MODEL COMPARISON — Test Set Results
======================================================================
           Model Accuracy Precision Recall F1-Score    AUC
             CNN   0.9693    0.9699 0.9693   0.9689 0.9958
          BiLSTM   0.9370    0.9407 0.9370   0.9379 0.9821
CNN+BiLSTM Hybrid   0.9710    0.9713 0.9710   0.9711 0.9922
======================================================================
```

Figure 6.7: Model Comparison Matrix

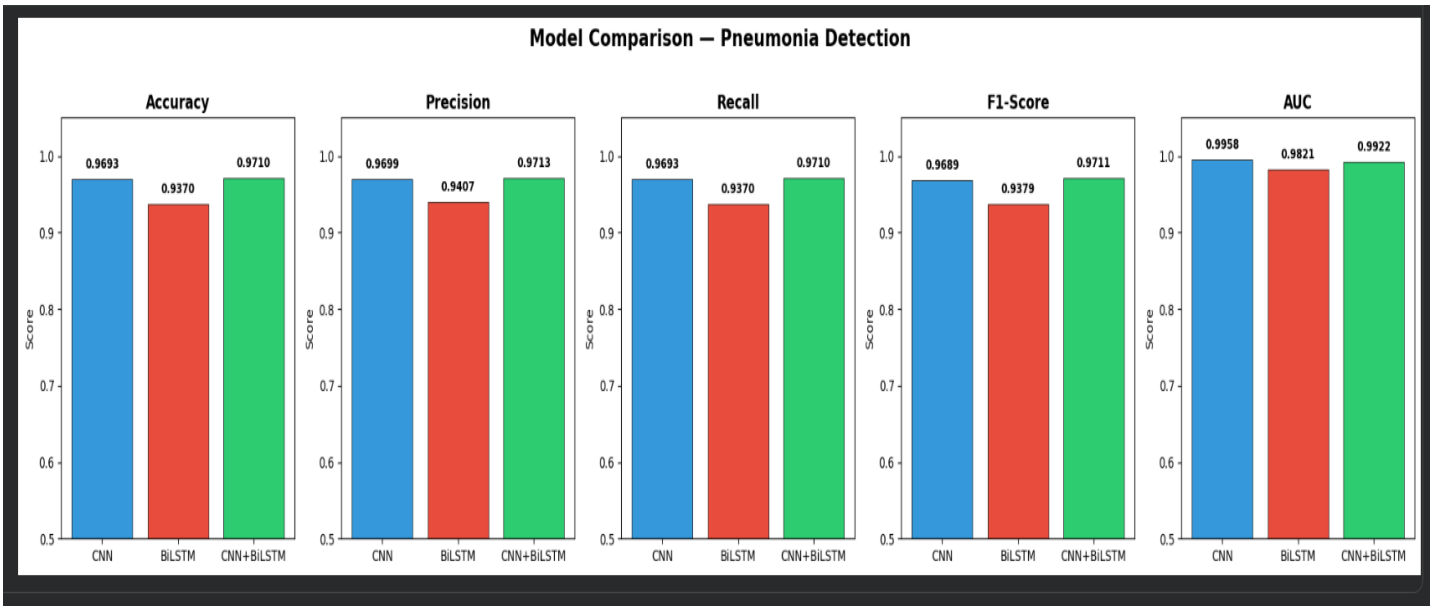

Figure 6.8: Model Comparison

CHAPTER # 7

7. Conclusion and Future Work

This chapter presents the overall conclusion of the PneumoAI project and highlights potential areas for future enhancements. It summarizes the achievements of the system, reflects on its effectiveness in addressing the identified problem, and outlines opportunities for further improvement and expansion.

7.1. Conclusion

The PneumoAI system was successfully designed and developed as an intelligent, web-based platform for the early detection of Pneumonia using chest X-ray images. The project effectively integrates advanced techniques from Artificial Intelligence with modern web technologies to deliver a comprehensive and user-friendly healthcare solution. By combining multiple system components into a unified platform, PneumoAI addresses key limitations of traditional diagnostic approaches, particularly in terms of accessibility and efficiency.

A major achievement of the system is the implementation of a deep learning model based on Convolutional Neural Networks, which enables automated classification of X-ray images as Normal or Pneumonia. The inclusion of confidence scores enhances transparency, allowing users to better interpret the reliability of predictions. This demonstrates the practical applicability of machine learning techniques in real-world medical imaging scenarios.

In addition to diagnostic capabilities, the integration of a conversational AI assistant powered by a Large Language Models significantly enhances the system's usability. The assistant provides context-aware guidance related to pneumonia, including symptoms, preventive measures, and recovery practices. This feature transforms the system from a purely analytical tool into an interactive platform that supports user education and engagement.

From a system design perspective, the adoption of a microservices-based architecture ensures modularity, scalability, and efficient communication between components such as the frontend, backend, machine learning service, and external APIs. This architectural approach allows independent development and scaling of different modules, improving maintainability and overall system performance. Additional features, including secure user authentication, prediction history tracking, PDF report generation, and administrative monitoring, further enhance the functionality and practicality of the system.

The system was rigorously evaluated through multiple testing approaches, including unit testing, functional testing, integration testing, and system-level validation. The results confirm that PneumoAI performs reliably under normal operating conditions, meeting defined requirements in terms of functionality, performance, and security. Error handling mechanisms and validation processes contribute to system robustness, ensuring consistent and stable operation.

Overall, PneumoAI demonstrates the effective application of AI technologies in healthcare, highlighting their potential to improve diagnostic support, increase accessibility, and enhance user awareness. While the system is not intended to replace professional medical diagnosis, it serves as a valuable assistive tool for early detection and patient education. The project reflects a successful integration of theoretical

knowledge and practical implementation, contributing to the advancement of AI-driven healthcare solutions.

7.2. Future Work

Although PneumoAI successfully achieves its primary objectives, there are several opportunities for enhancement that can further improve its capabilities and extend its real-world applicability. These potential improvements focus on increasing system accuracy, expanding functionality, and enhancing accessibility and user experience.

1. Multi-Disease Detection

The current system focuses on detecting Pneumonia; however, it can be extended to identify additional respiratory diseases such as tuberculosis, COVID-19, and lung cancer. Expanding the classification capabilities would transform PneumoAI into a more comprehensive diagnostic tool, increasing its usefulness in clinical and real-world scenarios.

2. Improved Model Accuracy

The performance of the deep learning model based on Convolutional Neural Networks can be further enhanced by training on larger and more diverse datasets. Advanced architectures such as ResNet and EfficientNet can be explored to improve accuracy and generalization. Additionally, techniques such as transfer learning and hyperparameter tuning can be applied to optimize model performance.

3. Mobile Application Development

To improve accessibility, a mobile version of the system can be developed. A dedicated mobile application would allow users to upload images, view results, and interact with the AI assistant directly from their smartphones. This would be particularly beneficial in remote or resource-limited areas where desktop access may be limited.

4. Real-Time Doctor Integration

Integrating telemedicine features would enable users to consult healthcare professionals directly through the platform. This could include real-time chat or video consultation, allowing users to validate AI-generated results with medical experts. Such integration would enhance trust and provide a more complete healthcare solution.

5. Explainable AI (XAI)

To improve transparency, the system can incorporate explainability features such as heatmaps or saliency maps that highlight regions of the X-ray influencing the prediction. This would help users and healthcare professionals understand how the model reaches its conclusions, increasing confidence in AI-driven decisions.

6. Multilingual Support

Adding support for multiple languages would make the system accessible to a broader audience. By enabling interaction in different languages, the platform can cater to users from diverse regions and backgrounds, improving inclusivity and usability.

7. Offline Capability

In areas with limited or unreliable internet connectivity, offline functionality can be introduced. This may involve deploying lightweight versions of the model on local devices, allowing basic predictions to be performed without requiring continuous internet access.

8. Enhanced Security Features

Given the sensitive nature of healthcare data, additional security measures can be implemented. These may include multi-factor authentication, data anonymization, and advanced encryption techniques to further protect user information and ensure compliance with data protection standards.

Overall Impact of Future Enhancements

The proposed improvements have the potential to significantly enhance PneumoAI in terms of accuracy, usability, scalability, and real-world applicability. By incorporating these features, the system can evolve from a prototype into a more robust and widely depl [5] [6] [7] [8] [9]oyable healthcare solution, contributing further to the advancement of AI-driven medical technologies.

8. References

[1] P. I. J. Z. K. Y. B. M. H. D. T. D. D. B. A. L. C. S. K. a. L. M. Rajpurkar, "CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep," 2017.

[2] D. Z. K. a. G. M. Kermany, "Identifying medical diagnoses and treatable diseases by image-based deep learning.," 2018.

[3] Qure.ai, "AI-based radiology solutions," 2024.

[4] Kaggle, "Chest X-Ray Pneumonia Dataset," 2024.

[5] I. S. a. G. E. H. A. Krizhevsky, ""ImageNet Classification with Deep Convolutional Neural Networks," Advances in Neural Information Processing Systems," 2012.

[6] P. F. a. T. B. O. Ronneberger, "“U-Net: Convolutional Networks for Biomedical Image Segmentation," in Proceedings of the International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)," 2015.

[7] K. C. G. C. a. J. D. T. Mikolov, "Efficient Estimation of Word Representations in Vector Space," arXiv preprint arXiv, 2013.

[8] T. B. e. al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS)," 2020.

[9] Y. B. a. A. C. I. Goodfellow, "Deep Learning," MIT Press, 2016.